

Stop Overthinking, Keep Passing the Tests

*Shortest-Correct LoRA Fine-Tuning versus Free Thinking Controls in a
Small Code Model (Qwen3.5-2B)*

Author: [your name] · **Supervisor:** [supervisor's name] · **[University], [Department]** ·
September 2026

Abstract

Some AI models "think" before they answer. They write notes to themselves first, then give the answer. Thinking often makes answers better, but it costs time and computer power. Small models are said to **overthink**: they think far longer than they need to.

This thesis asks one question: **on a small model with a thinking switch, is it better to *train the model to think shorter*, or to use a free option?** The free options are: switch thinking off, stop thinking at a fixed length, or ask the model in words to "think briefly".

We used the model **Qwen3.5-2B** and **234 code problems** from two well-known test sets (HumanEval+ and LiveCodeBench). We trained a small add-on (a **LoRA**) on the model's own shortest correct answers. Then we compared six ways of answering on the same problems, two tries each. Every answer was checked by running the test sets' own tests.

Main results.

- **The free thinking limit was the most accurate way.** Stopping thinking at 1,024 tokens solved **49.8%** of answers, against **42.1%** for normal thinking. That is +7.7 percentage points, and the 95% error bar [+4.3, +11.3] is fully above zero.
- **Training did not shorten thinking.** The trained model thought exactly as long as the normal model (x1.00). Its accuracy was 45.1%, a gain of +3.0 points that we cannot prove.
- **Thinking off was the cheapest.** It used 4× fewer tokens (860 against 3,446 per answer) at about the same accuracy as normal thinking.
- **Why:** the long answers were mostly **loops**. The model repeated the same lines until it ran out of room. Answers that finished were already short. Training on short correct answers cannot teach a model to get out of a loop. A thinking limit simply stops it.

Conclusion. For a small reasoning model on code, the waste is not long, careful thinking but **getting stuck**. A free thinking limit handles this better than training on the 2B model. We also ran the same 234 problems on **Qwen3.5-0.8B** and **Qwen3.5-4B**: the best free way was **OFF** (20.5%) on 0.8B, **limit 1,024** on 2B, and **limit 2,048** (78.2%) on 4B. LoRA-1 never beat that free winner. Still-larger models remain open future work.

Keywords: reasoning models, overthinking, thinking budget, LoRA, code generation, efficient inference.

The thesis in one page (for everyone)

The problem, with an everyday example

Imagine a student who writes five pages for every exam question, even "what is $2 + 2$ ". That is slow, and it doesn't make the answers better. New AI models do something similar. Before they answer, they "think" by writing notes to themselves. On easy code problems, these notes are often much longer than needed.

What we tried

Same model, same 234 code problems	┌	1. Thinking OFF	(switch it off)
	├	2. Thinking ON	(normal: what we compare against)
	├	3. "Think briefly"	(ask in words)
	├	4. Thinking limit	(stop thinking at 1,024 tokens)
	├	5. LoRA-1	(trained on short correct answers, small)
	└	6. LoRA-2	(trained on short correct answers, bigger)

Ways 1–4 are **free**: no training is needed. Ways 5–6 need training. We wanted to know: is training worth it?

What we found

Accuracy on 234 code problems (higher is better)

Thinking limit		49.8%	← best, and proven
LoRA-1		45.5%	
LoRA-2 (main)		45.1%	(not shorter)
Thinking ON		42.1%	(what we compare against)
Thinking OFF		40.8%	(4× cheaper)
"Think briefly"		6.6%	(confused the model)

Why it happened

We read the answers that were too long. Most of them were **loops**: the model wrote the same few lines again and again until it ran out of space. So the model was not "thinking too carefully". It was **stuck**.

- Our training showed the model short, correct answers. That teaches it nothing about how to get unstuck.
- A thinking limit simply stops the loop and makes the model answer. That works.

What it means

For small AI models that write code, **try free options first** (thinking OFF and a thinking limit). On Qwen3.5-2B, a **limit of 1,024** was best. On 0.8B, **OFF** was best. On 4B, a **limit of 2,048** was best. Training with LoRA-1 did not beat those free winners on any of the three sizes.

Key facts at a glance

Research question	Is training a small model to think shorter better than the free options?
Answer (2B)	No. A free thinking limit (1,024) was better.
Size check	Same 234 problems on 0.8B / 2B / 4B : best free = OFF / limit1024 / limit2048; LoRA-1 never won
Model (main)	Qwen3.5-2B (2 billion parameters, has a thinking on/off switch)
Test problems	234: HumanEval+ (164) and LiveCodeBench (70: 31 easy, 39 medium)
Answers checked	2,808 test answers on 2B (6 ways × 234 × 2 tries), plus size-extension runs on 0.8B and 4B
How answers were checked	By running each test set's own tests on the code (no checking by eye)
Best way (2B)	Thinking limit at 1,024 tokens: 49.8% , +7.7 points over normal thinking [+4.3, +11.3]
Cheapest way (2B)	Thinking off: 860 tokens per answer (normal thinking: 3,446)
Main reason	Long answers were mostly loops (69.5% of normal thinking's unfinished answers)
Computer used	Google Colab A100 GPU (main run ~38 units; size extension used further paid hours)

Contents

- 1. Introduction
 - 1.1 The problem
 - 1.2 Two ways to fix it
 - 1.3 The gap
 - 1.4 The research question
 - 1.5 What we did, in short
 - 1.6 What we found, in short
 - 1.7 Contributions
 - 1.8 How this thesis is organised
- 2. Background
 - 2.1 What is a large language model?
 - 2.2 What is a token?
 - 2.3 What is a reasoning model, and what is "thinking"?
 - 2.4 The thinking switch
 - 2.5 Overthinking, and a second problem: loops
 - 2.6 What is training, and what is LoRA?
 - 2.7 Training a model to think shorter: "shortest correct"
 - 2.8 The free options in earlier work
 - 2.9 Why small models may be different
 - 2.10 How code answers are checked: test sets
 - 2.11 Summary
- 3. Method
 - 3.1 The design in one picture
 - 3.2 The model
 - 3.3 The six ways of answering
 - 3.4 The test problems (234)
 - 3.5 The training data
 - 3.6 Making the training examples: "shortest correct"
 - 3.7 Training the LoRA
 - 3.8 The pilot study (the "mini-thesis")
 - 3.9 The fairness rules (fixed before the run)
 - 3.10 How the run was done
 - 3.11 What changed from the proposal, and why
 - 3.12 Ethics and safety
- 4. How We Measure: Where the Evidence Comes From
 - 4.1 Step 1: take the code out of the answer
 - 4.2 Step 2: run the benchmark's own tests
 - 4.3 Step 3: the pass rule
 - 4.4 Step 4: checking the checker
 - 4.5 Accuracy
 - 4.6 Points, not percent
 - 4.7 Comparing on the same problems ("paired")
 - 4.8 Error bars: is the difference real, or luck?
 - 4.9 Counting tokens, and "cut off"
 - 4.10 The thinking ratio
 - 4.11 Median: the middle value
 - 4.12 Finding loops
 - 4.13 Better or worse, problem by problem
 - 4.14 Checking whether the token limit was fair ("stage D")
 - 4.15 GPU time
 - 4.16 Where every number comes from

- 5. Results
 - 5.1 The headline: the free thinking limit was the most accurate way
 - 5.2 Accuracy of every way
 - 5.3 Cost: how many tokens each way used
 - 5.4 The hypotheses
 - 5.5 Unfinished answers and loops
 - 5.6 Problem by problem
 - 5.7 Was the token limit unfair to thinking ON? (stage D)
 - 5.8 Why "think briefly" failed
 - 5.9 Why LoRA-2 had little to learn from
 - 5.10 From the pilot study to the real test
 - 5.11 Computer time
 - 5.12 Summary of results
 - 5.13 Size extension: 0.8B and 4B (same family)
- 6. Analysis
 - 6.1 The answer to the research question
 - 6.2 Why training did not shorten thinking
 - 6.3 Why the thinking limit worked
 - 6.4 Thinking OFF is a strong, cheap option
 - 6.5 How this fits earlier work
 - 6.6 What could be wrong? (threats to validity)
 - 6.7 Size extension: what 0.8B and 4B change
 - 6.8 Summary of the analysis
- 7. Conclusion and Future Work
 - 7.1 The answer
 - 7.2 Main findings
 - 7.3 Advice for people who use small reasoning models for code
 - 7.4 What the hypothesis taught us
 - 7.5 Future work
 - 7.5.4 Size × limit extension — results (DECISIONS #72–#77)
 - 7.6 Closing
- References
- Appendices
 - Appendix A. Every problem, every number, every chart
 - Appendix B. How to repeat this work
 - Appendix C. How the project developed
 - Appendix D. Words used in this thesis

1. Introduction

1.1 The problem

AI models that write text, called **large language models** (LLMs), are now used every day to write computer code. A newer kind, the **reasoning model**, does something extra. Before it answers, it writes a **thinking** part: notes to itself, like a student's rough work on scrap paper. Then it writes the answer.

Thinking often helps. But it has a price. Every word the model writes costs time, electricity and money. Researchers have noticed that reasoning models often **overthink**. They write long thinking even for easy questions, where a short answer would be just as correct (Sui et al., 2025).

Everyday example. A student writes five pages for every exam question, even "what is $2 + 2$ ". The answers are not better, but the exam takes much longer.

This matters most for **small** models. Small models are cheap and can run on one ordinary graphics card (GPU). That is why people use them. If they waste most of their time on unneeded thinking, that advantage is lost.

1.2 Two ways to fix it

There are two kinds of fixes.

The free fixes need no training:

Free fix	How it works
Thinking OFF	Many 2026 models have a switch that turns thinking off. The model answers directly.
Thinking limit	Let the model think, but stop it after a fixed number of words and make it answer.
"Think briefly"	Ask the model in the question to keep its thinking short.

The trained fix teaches the model to think shorter by itself. A simple way is: let the model answer each practice problem several times, keep its **shortest answer that is still correct**, and train the model on those short answers (Munkhbat et al., 2025). Earlier work did this on math, and on code with larger models of 7–8 billion parameters, and reported 23–40% shorter thinking (SEER, 2025; ASAP, 2025).

1.3 The gap

The trained fix costs work and computer time. The free fixes cost nothing. So the key question is: **is training worth it compared with the free fixes, on the same model?**

We found no study that answers this for a small model that has a thinking switch, on code. Earlier training studies used models without a switch, so they could not compare with "thinking off". Studies of the switch (for example HRBench, 2026) compared ways of using the switch, but not a model trained to think shorter.

1.4 The research question

On a small model with a thinking switch, does training on its own shortest correct code answers give a better balance of accuracy and thinking length than the free options?

We expected yes. Our hypothesis had three parts. Compared with normal thinking, the trained model would:

1. **H1:** use at least **25% less** thinking;
2. **H2:** lose **no more than 3 points** of accuracy;
3. **H3:** be **more accurate** than thinking off, the thinking limit, and "think briefly".

All three rules were written down **before** we saw any result (Chapter 3). This matters: it stops us from choosing the rules later to fit the results.

1.5 What we did, in short

PROBLEM	→	GAP	→	QUESTION	→	HYPOTHESIS	→	EXPERIMENT	→	DATA	→	RESULTS	→	ANALYSIS	→	CONCLUSION
1.1		1.3		1.4		1.4		Ch. 3		Ch. 3		Ch. 5		Ch. 6		Ch. 7

- **Model:** Qwen3.5-2B, a small 2026 reasoning model with a thinking switch.
- **Test:** 234 code problems from HumanEval+ and LiveCodeBench.
- **Six ways of answering:** thinking ON, thinking OFF, "think briefly", a thinking limit, and two trained versions.
- **Checking:** every answer's code was run against the test sets' own tests. Chapter 4 explains exactly how we count a pass, compute accuracy, and decide whether a difference is real.

1.6 What we found, in short

The answer was **no**. Training did not make the model think shorter on the test problems. The free thinking limit was the most accurate way of all. The reason surprised us: the long answers were mostly **loops**, not careful thinking. Chapters 5 and 6 show the evidence.

1.7 Contributions

1. **A fair, same-model comparison** of a trained "think shorter" model against all three free options, on code, with rules fixed in advance.
2. **Evidence that the main waste in a small reasoning model is looping**, not long careful thinking, and that training on short correct answers does not remove it.
3. **Evidence that a simple thinking limit is a strong, free fix**, including a check that our token limit did not unfairly hurt normal thinking.
4. **Everything is open:** every raw answer, every grade, the scripts and the trained add-ons, so anyone can check or repeat the work.

1.8 How this thesis is organised

Chapter	What it covers
2. Background	What LLMs, tokens, thinking, LoRA and code test sets are; earlier work
3. Method	The six ways of answering, the data, the training, the fairness rules
4. How we measure	How a pass is decided, how accuracy, points, error bars and loops are computed
5. Results	All numbers, tables and charts (including the 0.8B / 4B size check)
6. Analysis	Why it happened, how it fits earlier work, and what could be wrong
7. Conclusion and future work	The answer, advice, size results, and what to test next

2. Background

This chapter explains the ideas the thesis builds on, from the ground up. If you already know how language models work, you can skip to Section 2.7.

2.1 What is a large language model?

A **large language model** (LLM) is a computer program that writes text. It works by guessing the next piece of text again and again.

Everyday example. Your phone's keyboard suggests the next word as you type. An LLM is the same idea, but much bigger and much better. It can continue a question with a whole correct answer.

The model learned to guess by reading a huge amount of text. What it learned is stored in billions of numbers called **parameters**. Our model, Qwen3.5-2B, has about **2 billion** parameters. That is small for 2026. It fits on one graphics card.

2.2 What is a token?

The model doesn't read or write whole words. It uses **tokens**: small pieces of text, about $\frac{3}{4}$ of a word on average. "Overthinking" might be split into "Over" + "think" + "ing".

Tokens are how we measure **cost**. The model writes one token at a time, so:

```
more tokens → more time → more electricity → more money
```

In this thesis, "the model wrote 3,446 tokens" means it wrote about 2,600 words.

2.3 What is a reasoning model, and what is "thinking"?

A **reasoning model** writes two parts:

```
Question → [ THINKING: notes to itself ... ] → [ ANSWER: the final code ]
```

The thinking part is like a student's rough work on scrap paper. It often makes the final answer more correct. In our model, the thinking ends with a special marker, `</think>`. Everything before the marker is thinking. Everything after it is the answer.

2.4 The thinking switch

Many 2026 models, including the Qwen3.5 family, have a **thinking switch**. With the switch ON, the model thinks before answering. With it OFF, the model answers directly. Qwen3.5-2B is OFF by default; we turned thinking on where needed with the setting `enable_thinking=True`.

2.5 Overthinking, and a second problem: loops

Overthinking means thinking much longer than needed, for example checking an easy answer again and again (Sui et al., 2025).

A related but different problem is the **loop** (also called *repetition*). The model starts repeating the same lines, and never reaches the end marker:

```
"...I'll track opening and closing parentheses... The implementation requires careful tracking...  
...I'll track opening and closing parentheses... The implementation requires careful tracking...  
...I'll track opening and closing parentheses..." ← until it runs out of space
```

This difference turns out to be the key to our results. Overthinking still ends with an answer. A loop never ends by itself. The makers of our model warn about exactly this. The official Qwen3.5-2B model card says: *"Qwen3.5-2B is more prone to entering thinking loops compared to other Qwen3.5 models, which may prevent it from terminating generation properly."*

2.6 What is training, and what is LoRA?

Training (or **fine-tuning**) means showing the model examples and nudging its parameters so it becomes more likely to write text like those examples.

Changing all 2 billion parameters is expensive. **LoRA** (Low-Rank Adaptation; Hu et al., 2022) is a cheaper way. It leaves the model unchanged and trains a **small add-on** that sits on top of it.

Everyday example. Instead of rewriting a whole textbook, you add sticky notes to some pages. The book stays the same; the notes change how you read it.

Our LoRA add-ons are about 44 MB, against 4.6 GB for the model.

2.7 Training a model to think shorter: "shortest correct"

The idea we test comes from Munkhbat et al. (2025):

1. The model answers each practice problem several times (here: 4 times).
2. Check each answer with the problem's tests.
3. Keep the SHORTEST answer that is still CORRECT.
4. Train the model on these short, correct answers.

The hope is that the model learns the habit: "stop thinking once you have enough".

What earlier work found:

Work	Setting	Result
Munkhbat et al., 2025	Math, models of 1.5–8B	About 12% fewer tokens with a plain prompt, accuracy about the same
SEER, 2025	Code, a 7B model	About 40% shorter thinking; LoRA about 7 points less accurate than full training
ASAP, 2025	Code, 7–8B models	23.5% fewer tokens on LiveCodeBench
S3-CoT, 2026	—	Warns that training on the <i>very</i> shortest answers hurts accuracy

None of these models had a thinking switch, so none could compare training with simply turning thinking off.

2.8 The free options in earlier work

- **Thinking OFF.** NoThinking (2025) showed that skipping thinking can beat thinking with a small budget, also on code. So "thinking off" is a serious option that any trained method must beat.
- **Thinking limit** (also called a *thinking budget* or *budget forcing*). Stop the thinking after a fixed number of tokens and make the model answer. The s1 paper (2025) describes budget forcing as "*forcefully terminating the model's thinking process*". The Qwen3 report (2025) also describes a "*thinking budget mechanism*".
- **Ask to be brief.** Put an instruction like "think briefly" in the question.
- **HRBench (2026)** compared many ways to use the thinking switch, on models from Qwen3.5-2B upwards, and found no single winner. It did not include a model trained to think shorter.

2.9 Why small models may be different

Two recent findings matter for a 2-billion-parameter model:

- **Small models learn less from long reasoning.** Li et al. (2025): "*small models ($\leq 3B$ parameters) do not consistently benefit from long chain-of-thought (CoT) reasoning or distillation from larger models.*"
- **Smaller models loop more.** Pipis et al. (2025): "*Larger models tend to loop less, and distilled students loop significantly even when their teachers rarely do.*"

We did not know these when we planned the study. They help explain our results (Chapter 6).

2.10 How code answers are checked: test sets

A **benchmark** is a fixed set of problems with a fair way to check answers. For code, the check is to **run the code against tests**. A **test case** is one input with the expected output:

```

Problem:   write add(a, b)
Test 1:    add(2, 3)    should give 5
Test 2:    add(-1, 1)   should give 0
Answer passes only if it passes ALL tests.
```

We used two benchmarks:

Benchmark	What the problems are like	Tests
HumanEval+ (Liu et al., 2023)	164 easy problems: complete one Python function	The original HumanEval tests extended " <i>by 80x</i> " with extra, harder tests
LiveCodeBench (Jain et al., 2024)	Programming-contest problems: read input, print output	Hidden tests from the contest site; new problems are added over time

LiveCodeBench keeps adding new problems. We used problems released from **February 2025 on**, to lower the chance that the model saw them while it was being built. We also used **MBPP+** (Liu et al., 2023), a set of easy Python problems, but only for training and for our pilot study, never as test problems in the main run.

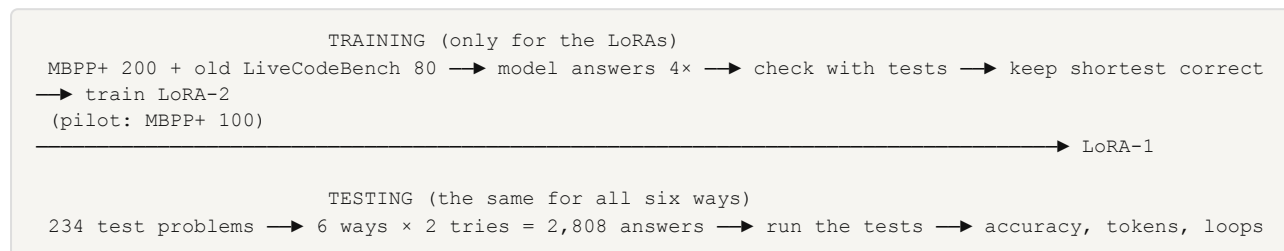
2.11 Summary

Reasoning models think before answering → thinking costs tokens → small models may overthink
→ or get stuck in LOOPS
Fixes: FREE (off / limit / "brief") vs TRAINED (shortest-correct LoRA)
Nobody had compared them on one small model with a switch, on code → this thesis

3. Method

This chapter says **what** we did and **why**. Chapter 4 explains **how we measured** it.

3.1 The design in one picture



The design is a **comparison on the same problems**: every way answers exactly the same 234 problems, with the same settings, so the only thing that changes is the way of answering.

3.2 The model

Model	Qwen3.5-2B (<code>unsloth/Qwen3.5-2B</code>), released February 2026, Apache-2.0 licence
Size	about 2 billion parameters; 4.6 GB in 16-bit
Why this model	Small, has a real thinking switch, free licence, and fast enough to finish in our time (Section 3.10)
Settings when answering	temperature 0.6, top-p 0.95, top-k 20 — the model makers' own settings for coding in thinking mode

Temperature, top-p and top-k control how random the model's word choices are. We used the official settings, so the model is not disadvantaged by our choices.

3.3 The six ways of answering

#	Way	Exactly what we did
1	Thinking ON	Switch on. The model thinks as long as it wants (up to the overall token limit). What we compare against.
2	Thinking OFF	Switch off. The model answers directly.
3	Think briefly	Switch on, and one sentence added to the question: <i>"Think briefly: keep your thinking to a few short sentences, then give the answer."</i>
4	Thinking limit	Switch on. After 1,024 thinking tokens we stop the thinking, add the end-of-thinking marker ourselves, and let the model write its answer.
5	LoRA-1	Switch on, plus the LoRA trained in the pilot study (100 MBPP+ training problems).
6	LoRA-2 (main)	Switch on, plus the LoRA trained for this run (200 MBPP+ + 80 older LiveCodeBench problems).

Why 1,024 for the limit? In the pilot study, the trained model thought for about 1,000 tokens. So the limit asks a fair question: *is a simple hard cut as good as training?*

Why LoRA-2 is the main one. We decided **before** the run that LoRA-2 would be the main result if it was trained. This stops us from picking whichever LoRA looks best afterwards.

3.4 The test problems (234)

Set	Problems	Level	What the model must do
HumanEval+	164	easy	Complete one Python function
LiveCodeBench	31	easy	Write a full program: read input, print output (a few are class methods)
LiveCodeBench	39	medium	The same, harder
Total	234		

- The LiveCodeBench problems were released from **February 2025 on**.
- **Hard problems were left out.** A 2-billion-parameter model solves almost none, so they could not show differences.
- The test set was **fixed on 2026-09-20, before any result**.

The question the model saw (identical for every way, except "think briefly"):

```
HumanEval+:      "Complete this Python function. Answer with one Python code block only,
                  containing the complete function." + the function to complete

LiveCodeBench:  the problem text + "Write a complete Python program. It reads from standard input
and
                  prints the answer to standard output. Answer with one Python code block only."
```

3.5 The training data

Pool	Problems	Why
MBPP+	200	Easy Python problems with good tests; the pilot showed they work
LiveCodeBench, older (release v1, before February 2025)	80 (40 easy + 40 medium)	The same <i>type</i> as the LiveCodeBench test problems, but older, so they can never be test problems

Overlap check. No test problem may appear in the training data. A script compared every training problem with every test problem and removed any that had the same function name, or shared many 5-word pieces of text (30% or more). It removed one training problem (`Mbpp/309`).

3.6 Making the training examples: "shortest correct"

```
For each training problem:
  1. Thinking ON answers it 4 times (token limit: 4,096 for MBPP+, 8,192 for LiveCodeBench).
  2. Each answer is checked with the problem's tests.
  3. From the correct answers, keep the SHORTEST one ...
  4. ... but never one shorter than HALF of the middle (median) correct length for that problem.
```

Why step 4? The S3-CoT paper warns that training on the very shortest answers can hurt accuracy. A lucky, too-short answer would teach "always stop early". So we keep the shortest *reasonable* answer.

The kept answer is stored **word for word**, with its thinking and its code, and becomes one training example.

3.7 Training the LoRA

Setting	Value
Method	LoRA on the attention and feed-forward layers
LoRA size	rank 16, alpha 16
Learning rate	0.0002, with a short warm-up
Epochs (passes over the data)	3
Batch	1 example $\times$ 4 steps gathered together
Seed	3407
Tool	Unsloth, following its Qwen3.5 guide

LoRA-2 trained on **157 examples** (133 from MBPP+, 24 from LiveCodeBench) in about **6 minutes**.

A safety check on the add-on. In the pilot we found that the saved LoRA could load "empty", with its parts matched to nothing, so the "trained" model silently behaved like the untrained one. The answering script now **stops** if any LoRA part is not matched to the model. This check passed for both LoRAs.

3.8 The pilot study (the "mini-thesis")

Before the main run, we ran the whole method once, small:

- 100 MBPP+ training problems $\rightarrow$ shortest-correct examples $\rightarrow$ LoRA-1.
- Tested on **100 other MBPP+ problems**, 1 try each.

Pilot result: LoRA-1 used 41% fewer tokens than thinking ON (x0.59) **and** was more accurate (65% against 50%). This was a strong sign that the method works, so we went ahead with the main run. The main run then tested whether this holds on **different** problems (Chapter 5.10).

3.9 The fairness rules (fixed before the run)

Rule	Value	Why
Same problems for every way	all 234	a fair, paired comparison
Same overall token limit for every way	4,096 (HumanEval+), 8,192 (LiveCodeBench)	nobody gets more room
Same settings and seeds	temperature 0.6, top-p 0.95, top-k 20; a fixed seed per try	only the way of answering changes
Same prompt	except the one extra sentence for "think briefly"	
Tries per problem	2	budget (Section 3.10)
Main LoRA	LoRA-2	fixed in advance
Hypotheses and their thresholds	H1–H3 (Section 1.4)	fixed in advance
Grading	the benchmarks' own tests, never by eye	Chapter 4
Raw answers	every answer saved word for word	anyone can re-check

Why 4,096 and 8,192? HumanEval+ problems are easy; in the pilot almost every answer that reached 4,096 was a loop. LiveCodeBench has medium problems, where honest reasoning can be longer. To check that these limits did not unfairly hurt normal thinking, we re-ran thinking ON's cut-off answers with **16,384** tokens (Section 4.9).

3.10 How the run was done

Stage	What	Colab units (estimated)
Smoke test	every way on 2 problems per set, to check that everything works	0.6
A	5 ways × 234 problems, try 1	2.9 on 24 Sept, plus part of the 21 Sept session (that session used about 8.7 units in total, including idle time)
B	make LoRA-2's training data, train LoRA-2, test it (try 1)	9.0
C	try 2 for all 6 ways	10.3
D	re-run thinking ON's cut-off answers with 16,384 tokens	4.2

- **Computer:** one Google Colab **A100** GPU, bfloat16 numbers, batches of up to 128 answers at a time.
- **Budget rule:** a stage started only if at least 20 units would be left afterwards.
- **Total:** about 38 units, from 2026-09-21 to 2026-09-24 (including set-up time).
- Results were saved to Google Drive after every step, so a disconnect lost nothing.

3.11 What changed from the proposal, and why

We report every change honestly. All were made **before** the main results were seen.

Proposal	What we did	Why
Model Gemma-4-E4B	Qwen3.5-2B	Gemma ran at only about 4.4 tokens per second on a free GPU; far too slow to finish. Qwen3.5-2B is smaller and has the same kind of switch.
Free GPUs only (\$0)	A small paid Colab package (A100)	A one-week deadline; free GPUs were too slow.
About 1,000 test problems incl. MBPP+	234 (HumanEval+ + LiveCodeBench)	1,000 problems $\times$ 5 ways $\times$ 4 tries would need 13+ GPU hours.
4 tries per problem	2	Budget. Cost: wider error bars.
Training on DeepCoder and APPS	MBPP+ + older LiveCodeBench	Already loaded and graded by tested code; less risk in one week.
Five ways of answering	Six (two LoRAs)	LoRA-2 added medium-style training problems; LoRA-1 kept for comparison.

3.12 Ethics and safety

- The model's code was **never run inside our notebook**. It ran in a separate process, in a temporary folder, with a time limit, so broken or harmful code could not damage anything.
- No personal data was used. All data sets are public.

4. How We Measure: Where the Evidence Comes From

Every number in this thesis comes from one of the steps below. This chapter explains each step with a small worked example, so anyone can check our numbers by hand.

```
answer text → take out the code → run the tests → PASS or FAIL → accuracy → compare two ways →  
error bar  
    └──→ count tokens (thinking / answer) → cost → cut off? loop?
```

4.1 Step 1: take the code out of the answer

The model's answer is text. We look for the **last Python code block** in the answer part (the text after the end-of-thinking marker `</think>`). That code is what we test.

If the model never reached `</think>`, there is **no answer part**, so there is no code to test. That answer is a **fail** (Section 4.9).

4.2 Step 2: run the benchmark's own tests

We never judge code by reading it. We **run** it against the benchmark's own tests:

	HumanEval+	LiveCodeBench
What a test is	A call like <code>has_close_elements([1.0, 2.8, 3.0], 0.3)</code> with the correct result	An input text fed to the program, and the exact output it must print
Which tests	All of HumanEval+'s tests (the original ones extended "by 80x", Liu et al., 2023)	The problem's public and hidden tests; we use up to 20 per problem to keep grading fast
How output is compared	The test program checks the result itself	Line by line, ignoring spaces at the end of lines
Time limit	30 seconds for the whole test program	20 seconds per test

Safety. The code runs in a **separate process**, in a temporary folder, with the time limit. It never runs inside our notebook.

4.3 Step 3: the pass rule

An answer PASSES only if it passes EVERY test. One failed test = FAIL.

An answer fails if any of these happen:

Why it failed	Real example from our data
Wrong result on a test	wrong answer on test 1 (the most common reason: 337 times)
The code crashes	<code>NameError: name 'List' is not defined</code> (a missing import) · <code>SyntaxError: invalid syntax</code>
The code is too slow	<code>timeout</code> (13 times in all 2,808 answers)
No code at all (cut off)	the answer never finished thinking; the test then can't find the function, e.g. <code>NameError: name 'separate_paren_groups' is not defined</code>

This is strict, and the same for every way.

4.4 Step 4: checking the checker

A broken checker would make every number wrong. So **before** grading any model answer, the notebook graded the benchmarks' **official correct solutions** for 30 HumanEval+ and 20 MBPP+ problems. All had to pass, or the run would stop. They all passed.

4.5 Accuracy

Accuracy = answers that passed ÷ all answers

Worked example (thinking ON, all 234 problems):

```
234 problems × 2 tries = 468 answers
passed: 197
accuracy = 197 ÷ 468 = 0.421 = 42.1%
```

Every problem has exactly 2 tries, so every problem counts equally.

4.6 Points, not percent

When we compare two accuracies, we use **percentage points** ("points"):

```
Thinking limit 49.8% - Thinking ON 42.1% = +7.7 points
```

This is **not** "+7.7 percent". In relative terms, $7.7 \div 42.1$ is an 18% improvement. We always report **points**, because they are simpler and cannot be inflated by a small starting value.

4.7 Comparing on the same problems ("paired")

Some problems are easy and some are hard. If one way happened to get easier problems, it would look better for no real reason. So we always compare two ways **on the same problems**, one problem at a time.

For each problem, each way's score is the share of its 2 tries that passed (0, 1/2 or 1). Then:

Problem	Thinking ON	Thinking limit	difference (points)
HumanEval/12	1/2 = 50%	2/2 = 100%	+50
HumanEval/40	0/2 = 0%	0/2 = 0%	0
lcb/3705	0/2 = 0%	1/2 = 50%	+50
...	...	...	...
average over all 234 problems			+7.7 points

(The three rows are made-up examples to show the idea. The +7.7 is the real average.)

4.8 Error bars: is the difference real, or luck?

With only 234 problems and 2 tries each, some difference can appear just by luck. An **error bar** (a 95% interval) shows how big the difference could reasonably be.

We compute it with the **bootstrap**, a method that uses the data itself:

1. Take our 234 problems.
2. Draw 234 problems AT RANDOM from them, WITH replacement (so some problems appear twice and some not at all).
3. Compute the average difference on this new set.
4. Repeat 2,000 times. → 2,000 slightly different averages
5. Sort them. The 50th and the 1,950th values are the error bar (the middle 95%).

Tiny example with 5 problems. Differences: +50, 0, 0, +100, -50. Average = +20.

```
random draw 1: +50, +50, 0, -50, +100 → average +30
random draw 2: 0, 0, -50, -50, 0 → average -20
random draw 3: +100, +50, +100, 0, 0 → average +50
... 2,000 draws ... sort ... keep the middle 95%
```

With only 5 problems the draws jump around a lot, so the error bar would be very wide. With 234 problems it is much narrower.

The rule we use:

If the whole error bar is **above 0**, the way is **proven better** on this test set. If it is **below 0**, it is proven worse. If it **includes 0**, we cannot tell.

Real example: thinking limit vs ON = +7.7 [+4.3, +11.3]. The whole bar is above 0, so the limit is proven better. LoRA-2 vs ON = +3.0 [-1.1, +7.3]. The bar includes 0, so we cannot say LoRA-2 is better.

The random draws use a fixed **seed** (3407), so running the script again gives exactly the same error bars.

4.9 Counting tokens, and "cut off"

We count tokens with the model's own tokenizer:

```
[ thinking tokens ..... ] </think> [ answer tokens ]
      thinking                answer part
|----- all tokens -----|
```

- **Thinking tokens:** everything before `</think>`.
- **Answer part:** everything after it.
- **All tokens:** both together. This is the real cost, because the model has to write all of it.

Cut off. Every way had the same overall limit: 4,096 tokens (HumanEval+) or 8,192 (LiveCodeBench). If an answer reaches the limit before it ends, it is **cut off**. If it was still thinking, **all** its tokens count as thinking and there is no code, so it **fails**. The rule is the same for every way.

4.10 The thinking ratio

To compare thinking length, we divide:

```
thinking ratio = total thinking tokens of a way ÷ total thinking tokens of thinking ON (same
problems)
```

- **x1.00** = the same as ON.
- **x0.75** = 25% shorter (the target in hypothesis H1).
- **x0.26** = 74% shorter.

It gets an error bar in the same way as accuracy (Section 4.8).

Watch out: the thinking limit stops thinking at 1,024 tokens, but the model can keep reasoning inside its answer part. So for the limit we also report **all tokens**, which gives the honest saving (Chapter 5.3).

4.11 Median: the middle value

A few very long answers can pull an average up a lot. So we also report the **median**: sort all the values and take the middle one.

```
lengths 500, 600, 700, 800, 4,096 → average 1,339, median 700
```

We use the median of **finished** answers (not cut off) to show how long normal, successful thinking is.

4.12 Finding loops

To check whether a cut-off answer is a **loop**, we use a simple rule:

Take a piece of the answer's **last 200 characters**. If that exact piece already appears **at least twice** earlier in the same answer, the answer is a loop.

```

"... I'll track opening and closing parentheses ... [same text] ... [same text] ... [same text]"
                                                                    ▲ last 200 characters
                                                                    also found twice earlier →
LOOP

```

This rule is **strict**. It misses loops where the model repeats with small changes. So our loop counts are a **lower bound**: the real number is probably higher.

4.13 Better or worse, problem by problem

For each problem we also ask: did a way solve **more** of its 2 tries than thinking ON, **fewer**, or the **same**? Counting problems gives a simple picture that doesn't depend on averages (Chapter 5.6).

4.14 Checking whether the token limit was fair ("stage D")

A fixed limit might cut off long but useful thinking. To test this, we took every thinking-ON answer from try 1 that was cut off, and ran it again with **16,384** tokens, four times the room. If many of them then finish and pass, the limit was unfair to thinking ON. Chapter 5.7 gives the answer.

4.15 GPU time

We also record how long the GPU took. Each batch of answers has a measured time, which we share equally among its answers. But a batch waits for its **slowest** answer, so GPU time mostly reflects how we grouped answers. That is why we use **tokens**, not minutes, as the measure of cost.

4.16 Where every number comes from

Evidence	File
Every raw answer, word for word (thinking + code)	results/2026-09-24-thesis-run/test-*.jsonl
Every grade (passed, why not, tokens, cut off)	results/2026-09-24-thesis-run/test-*.graded.csv
Error bars printed by the run	results/2026-09-24-thesis-run-step12-output.txt
All tables, charts and problem lists	results/full-results/FULL-RESULTS.md
The scripts that compute them	scripts/compare_thesis.py, scripts/make_full_results.js, scripts/check_thesis_run.js

Anyone can re-grade or re-count everything from the raw answers, without a GPU.

5. Results

This chapter gives the numbers, without opinions. Chapter 6 explains them. Chapter 4 explains how each number was computed. The full tables, every problem, and all charts are in [results/full-results/FULL-RESULTS.md](#).

What was run: 6 ways $\times$ 234 problems $\times$ 2 tries = **2,808 test answers**, all checked with the benchmarks' own tests, on one Colab A100 GPU (2026-09-21 to 2026-09-24).

5.1 The headline: the free thinking limit was the most accurate way

Stopping the model's thinking at 1,024 tokens gave the highest accuracy of all six ways: 49.8%, against 42.1% for normal thinking. This is +7.7 points, and the error bar [+4.3, +11.3] is fully above zero.

The evidence, all on the same 234 problems:

#	Evidence	Number	Proven?
1	More accurate than normal thinking	+7.7 points [+4.3, +11.3]	✓ yes
2	More accurate than our trained model (LoRA-2)	+4.7 points [+0.2, +9.0]	✓ yes
3	More accurate than thinking OFF	+9.0 points [+3.8, +14.3]	✓ yes
4	Rarely makes a problem worse	better than ON on 38 problems, worse on only 12	—
5	Far fewer unfinished answers	25.9% cut off, against 40.6% for normal thinking	—
6	Cheaper than normal thinking	2,722 tokens per answer, against 3,446 (21% fewer)	—
7	Works on easy problems of both test sets	HumanEval+ +5.2 [+1.2, +9.1]; LiveCodeBench easy +32.3 [+17.7, +46.8]	✓ yes, both

Where it does not help: on LiveCodeBench **medium** problems it solved nothing (0 of 78 answers). Stage D (Section 5.7) also shows that part of its advantage on HumanEval+ comes from our 4,096-token limit.

5.2 Accuracy of every way

Table 5.1. Accuracy (answers passed $\div$ answers). "vs ON" is the difference from thinking ON in points, with the 95% error bar.

Way	All 234	vs ON	HumanEval+ (164)	LCB easy (31)	LCB medium (39)
Thinking limit	49.8% (233/468)	+7.7 [+4.3, +11.3]	60.4%	56.5%	0.0%
LoRA-1	45.5% (213/468)	+3.4 [-0.4, +7.7]	56.4%	41.9%	2.6%
LoRA-2 (main)	45.1% (211/468)	+3.0 [-1.1, +7.3]	56.4%	38.7%	2.6%
Thinking ON	42.1% (197/468)	—	55.2%	24.2%	1.3%
Thinking OFF	40.8% (191/468)	-1.3 [-6.6, +4.3]	47.3%	46.8%	9.0%
Think briefly	6.6% (31/468)	-35.5 [-41.2, -29.3]	3.7%	30.6%	0.0%

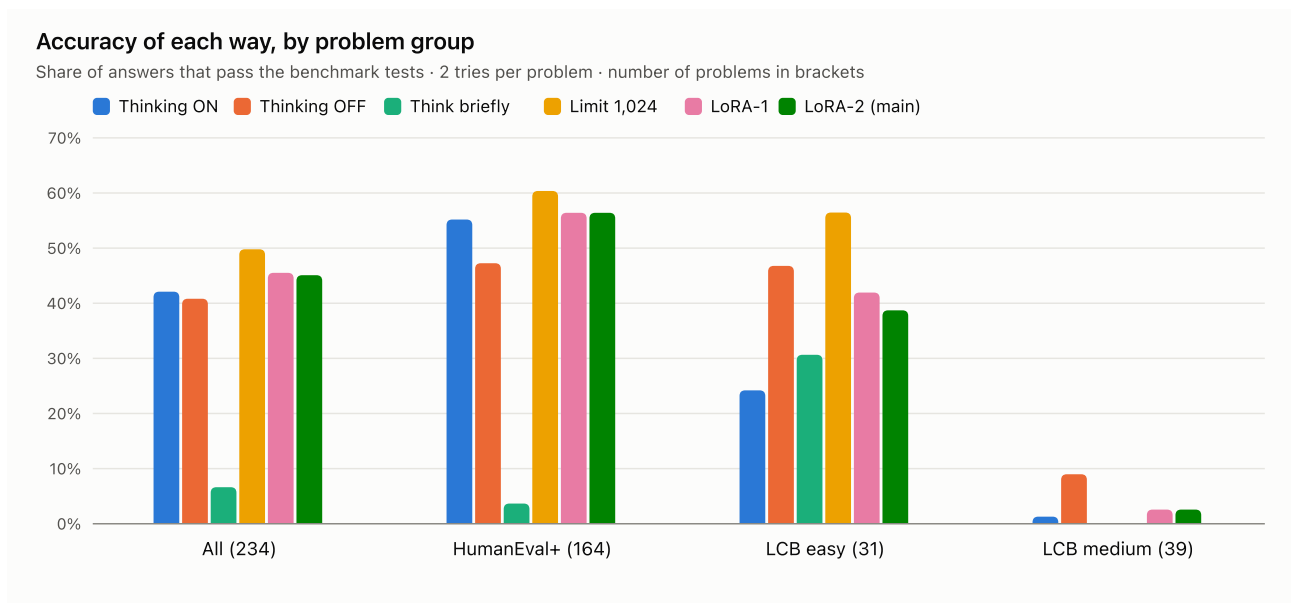


Figure 5.1. Accuracy of each way, by problem group.

Differences by group (vs thinking ON):

Way	HumanEval+	LCB easy	LCB medium
Thinking limit	+5.2 [+1.2, +9.1]	+32.3 [+17.7, +46.8]	-1.3 [-3.8, 0.0]
LoRA-1	+1.2 [-3.7, +6.4]	+17.7 [+3.2, +32.3]	+1.3 [-2.6, +5.1]
LoRA-2	+1.2 [-4.3, +6.7]	+14.5 [+3.2, +29.0]	+1.3 [-2.6, +5.1]
Thinking OFF	-7.9 [-14.6, -1.2]	+22.6 [+8.1, +37.1]	+7.7 [+1.3, +15.4]
Think briefly	-51.5 [-57.9, -44.8]	+6.5 [-6.5, +19.4]	-1.3 [-3.8, 0.0]

Bold = proven (the error bar doesn't include 0).

- On **HumanEval+**, thinking helps: turning it off costs 7.9 points.
- On **LiveCodeBench easy**, normal thinking is weak (24.2%). Every way except "brief" is clearly better.
- On **LiveCodeBench medium**, every way is near zero. Only thinking OFF solves a few (9.0%).

Extra comparisons (computed with the same method after the run; not part of the planned hypotheses):

Comparison	All	HumanEval+	LCB easy	LCB medium
Limit – OFF	+9.0 [+3.8, +14.3]	+13.1 [+6.4, +19.2]	+9.7 [−4.8, +24.2]	−9.0 [−16.7, −2.6]
Limit – LoRA-2	+4.7 [+0.2, +9.0]	+4.0 [−1.5, +9.5]	+17.7 [+1.6, +33.9]	−2.6 [−6.4, 0.0]
LoRA-2 – LoRA-1	−0.4 [−4.5, +3.8]	0.0 [−5.5, +5.2]	−3.2 [−16.1, +9.7]	0.0 [−3.8, +3.8]

LoRA-2 and LoRA-1 were the same: the extra training data did not help.

5.3 Cost: how many tokens each way used

Table 5.2. Tokens per answer, all 234 problems.

Way	All tokens (average)	Thinking (average)	Answer part (average)	Thinking of finished answers (median)	Thinking vs ON
Thinking OFF	860	0	860	0	x0.00
Thinking limit	2,722	843	1,879	880	x0.26 [0.23, 0.29]
LoRA-1	3,158	2,838	320	740	x0.87 [0.81, 0.93]
LoRA-2 (main)	3,392	3,252	140	719	x1.00 [0.94, 1.06]
Thinking ON	3,446	3,262	185	751	—
Think briefly	5,080	5,064	16	3,547	x1.55 [1.43, 1.70]

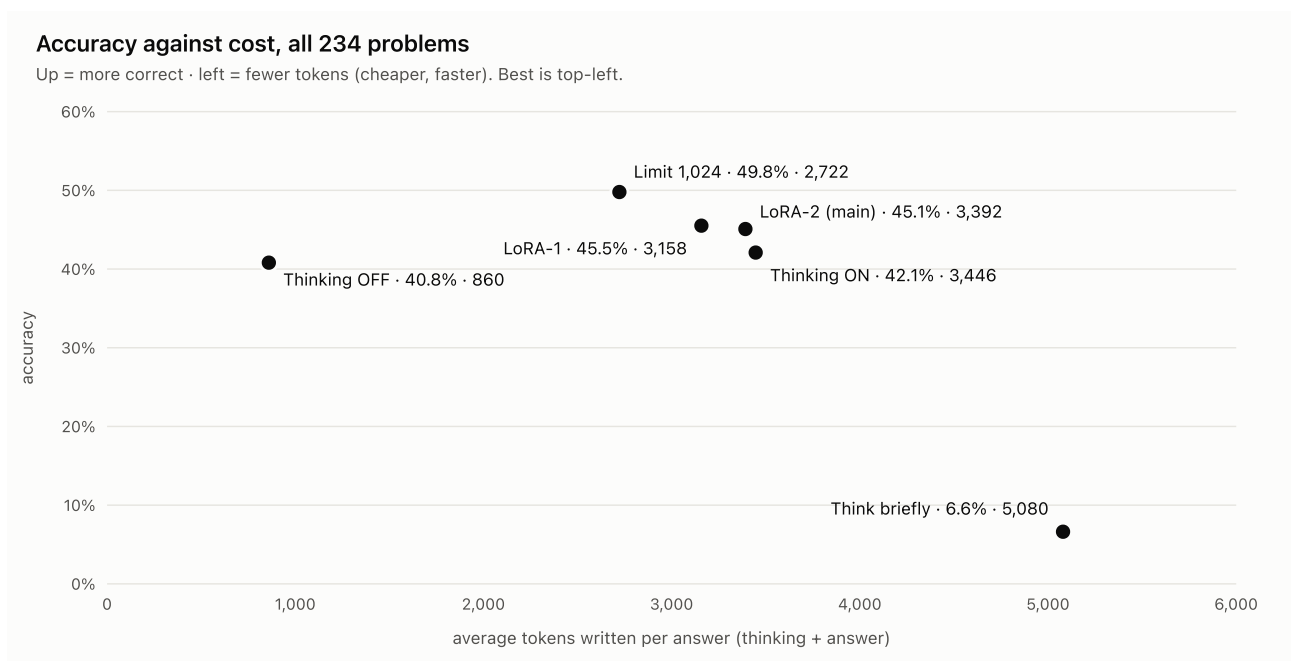


Figure 5.2. Accuracy against tokens per answer. The best place is top-left: accurate and cheap.

- LoRA-2 did not shorten thinking at all (x1.00).** LoRA-1 shortened it a little (x0.87), mostly on LiveCodeBench easy (x0.67).

2. **The limit keeps writing after the cut.** Its thinking is 74% shorter than ON's, but its answer part is ten times longer (1,879 against 185 tokens). Counted honestly, over all tokens, it saves **21%**.
3. **Thinking OFF is by far the cheapest:** 4× fewer tokens than ON.
4. **"Think briefly" made thinking longer**, not shorter (x1.55).

5.4 The hypotheses

The hypotheses were fixed before the run, for the main trained model, LoRA-2.

Table 5.3. The hypotheses.

	Hypothesis	Needed	Result	Verdict
H1	Thinking at least 25% shorter than ON	$\leq x0.75$	x1.00 [0.94, 1.06]	 Rejected
H2	Accuracy at most 3 points below ON	≥ -3	+3.0 [-1.1, +7.3]	 Supported
H3a	More accurate than thinking OFF	> 0	+4.3 [-0.6, +9.2]	 Not shown (bar includes 0)
H3b	More accurate than the thinking limit	> 0	-4.7 [-9.0, -0.2]	 Rejected: the limit is better
H3c	More accurate than "think briefly"	> 0	+38.5 [+32.9, +44.2]	 Supported (but see 5.8)

The overall hypothesis is rejected: H1 and H3b fail.

5.5 Unfinished answers and loops

Many thinking answers never finished. We checked each unfinished answer for a **loop** (Section 4.12).

Table 5.4. Cut-offs and loops (468 answers per way).

Way	Cut off	Of those, loops
Thinking OFF	33 (7.1%)	27 (81.8%)
Thinking limit	121 (25.9%)	93 (76.9%)
LoRA-1	168 (35.9%)	141 (83.9%)
Thinking ON	190 (40.6%)	132 (69.5%)
LoRA-2 (main)	192 (41.0%)	170 (88.5%)
Think briefly	432 (92.3%)	174 (40.3%)

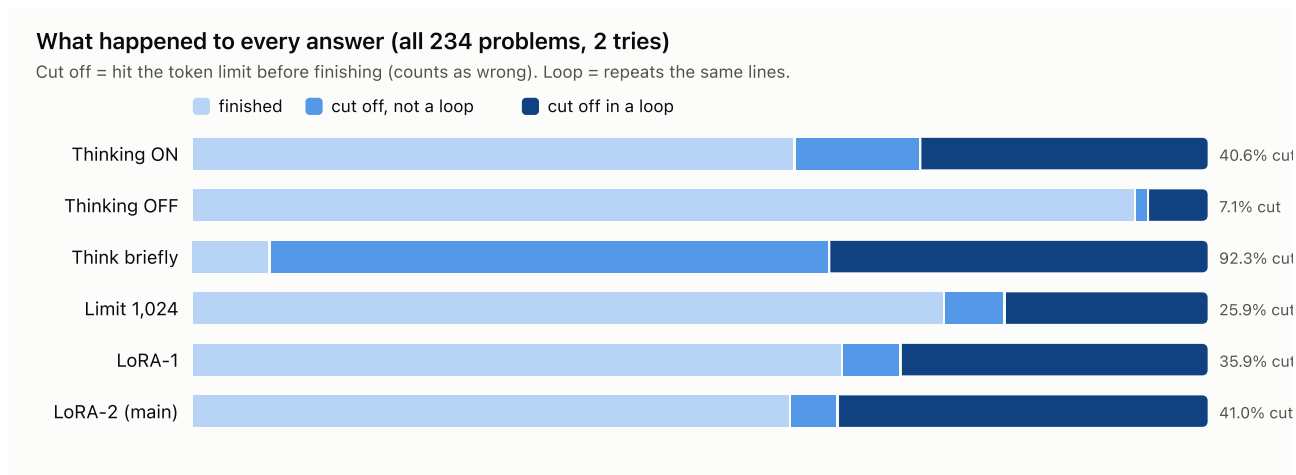


Figure 5.3. Finished, cut off in a loop, or cut off without a loop.

Finished answers were short. On HumanEval+, the median thinking of answers that finished was **672** tokens for ON, **711** for LoRA-1 and **695** for LoRA-2. Up to about 1,000 tokens, the three curves in Figure 5.4 almost lie on top of each other. They differ mainly in how many answers **never** finish.

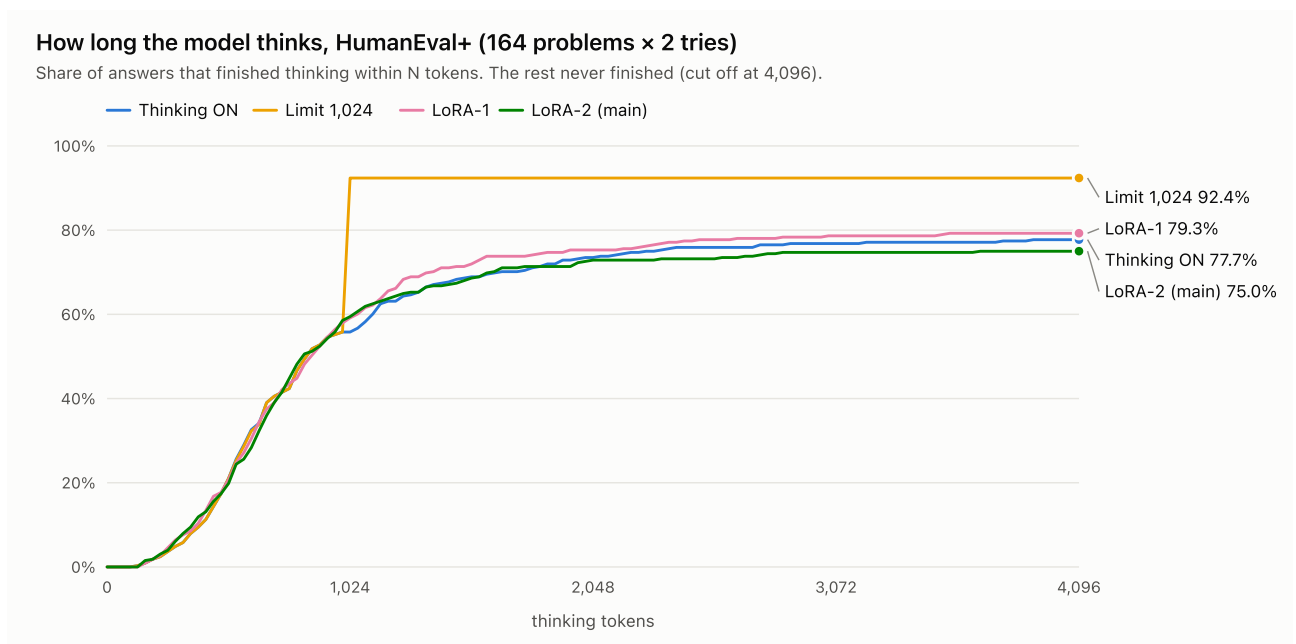


Figure 5.4. Share of HumanEval+ answers that finished thinking within N tokens.

A real loop (thinking ON, HumanEval/1, the end of an answer cut off at 4,096 tokens):

```
I'll track opening and closing parentheses, ensuring each group is properly balanced and separated.
The
algorithm needs to handle potential edge cases like empty strings and ensure correct grouping.

The implementation requires careful tracking of parentheses groups, checking for balance, and
collecting
valid groups into a list.

By iterating through the string, I can identify balanced groups without nested structures, ...
[the same three paragraphs repeat until the limit]
```

5.6 Problem by problem

Table 5.5. Problems (of 234) where each way solved more of its 2 tries than thinking ON ("better"), fewer ("worse"), or the same.

Way	Better	Worse	Same
Thinking limit	38	12	184
LoRA-2 (main)	45	29	160
LoRA-1	44	31	159
Thinking OFF	45	40	149
Think briefly	8	114	112

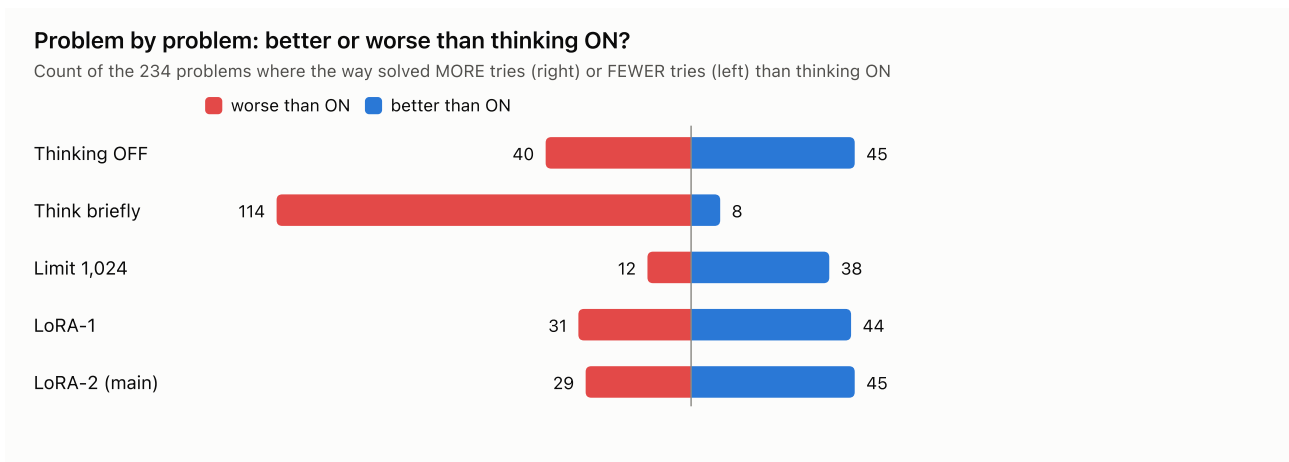


Figure 5.5. Problem by problem, against thinking ON.

The limit had the best balance: it made only 12 problems worse. Across all ways, **168** of the 234 problems were solved at least once; **66** were solved by no way at all.

5.7 Was the token limit unfair to thinking ON? (stage D)

We gave every cut-off thinking-ON answer from try 1 four times more room (16,384 tokens).

Table 5.6. Thinking ON with 16,384 tokens for its cut-off answers (try 1 only).

	HumanEval+	LiveCodeBench
Cut-off answers re-run	36	60
Now finished	23	10
Now correct	9	5
Still cut off, in a loop	12	48
Thinking ON, normal limit	53.7%	10.0%
Thinking ON, with 16,384 tokens	59.1%	17.1%
Thinking limit (try 1)	60.4%	25.7%
Thinking OFF (try 1)	48.2%	28.6%

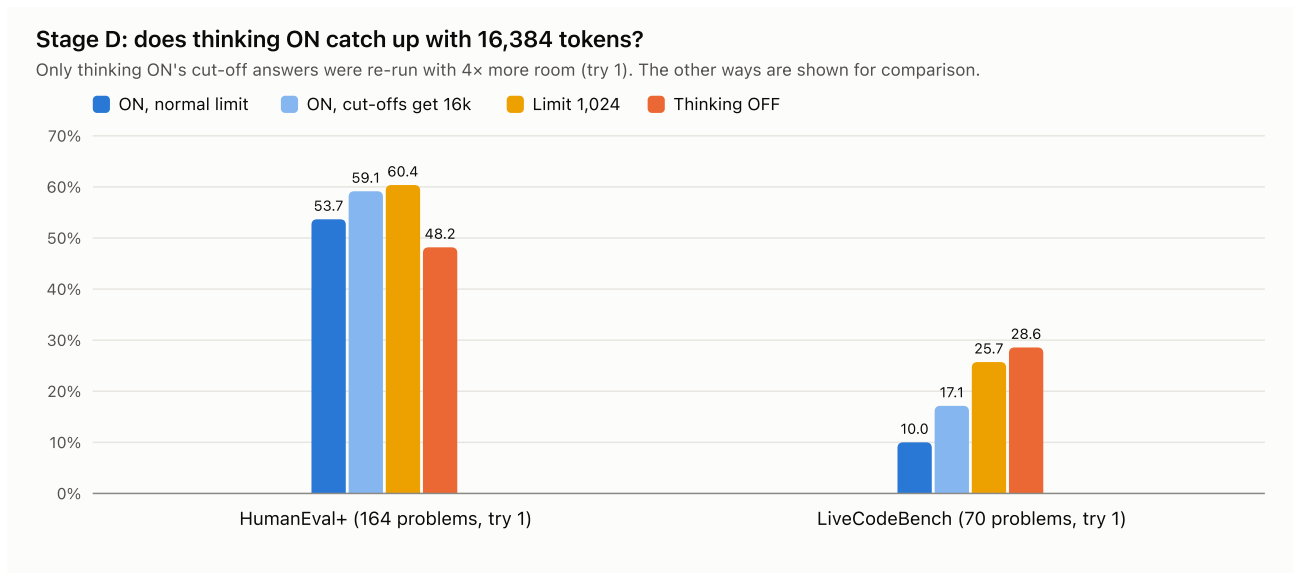


Figure 5.6. Thinking ON with four times more room, against the limit and OFF.

- **HumanEval+:** with more room, thinking ON rose by 5.5 points (9 more of 164 problems), close to the limit's 60.4%. So the 4,096 limit did cost thinking ON something here, and part of the limit's advantage on HumanEval+ comes from it. But thinking ON needed up to 16,384 tokens to get there; the limit needs far fewer.
- **LiveCodeBench:** more room helped little. 48 of the 50 answers still cut off were loops. Thinking ON stayed below both the limit and OFF.

5.8 Why "think briefly" failed

Only 36 of 468 "think briefly" answers finished. Our question already said *"Answer with one Python code block only."* The extra sentence said *"Think briefly ... then give the answer."* The model treated these as two rules that clash, and argued about them until it ran out of space (HumanEval/o):

```
* "Answer with one Python code block only" suggests I should not include conversational filler.
* "Think briefly... then give the answer." suggests I should include the thinking.
...
* Wait, if I output thinking text, is it "one Python code block only"?
```

This result is about **this wording** combined with our answer rule. It does not show that asking to be brief never works.

5.9 Why LoRA-2 had little to learn from

Table 5.7. From training problems to training examples (4 thinking-ON tries per problem).

	MBPP+	LiveCodeBench (older)
Problems	200	80 (40 easy, 40 medium)
Correct answers	357 of 800 (44.6%)	44 of 320 (13.8%); medium: 1 of 160
Problems with ≥ 1 correct	134	24
Problems with ≥ 2 correct (a real choice of "shortest")	111	12
Kept as training examples	133	24
Median thinking of kept examples	515	4,709
Kept length $\div$ average correct length (lower = more to learn)	0.829	0.987

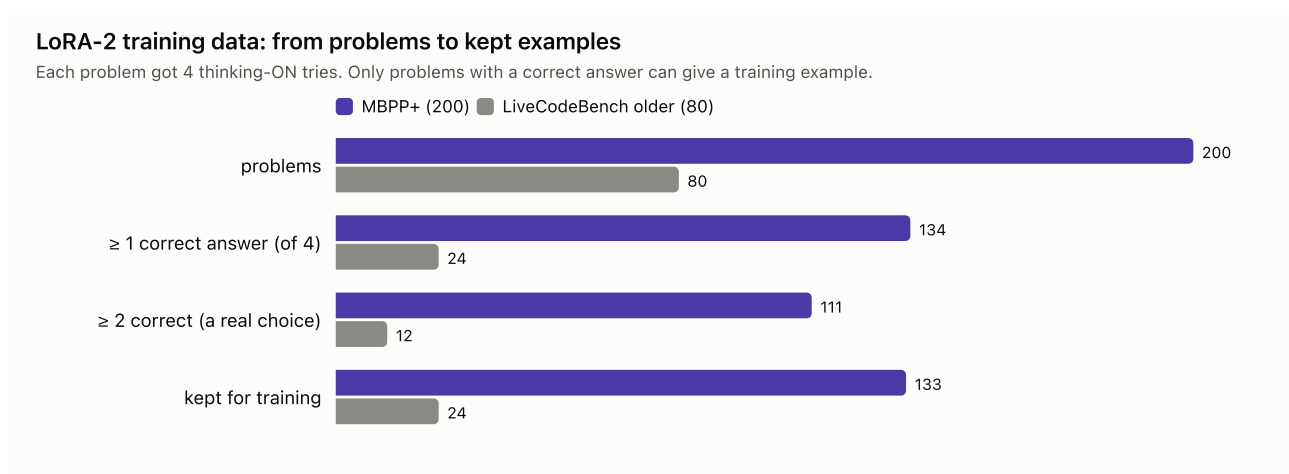


Figure 5.7. From problems to kept training examples.

- The medium problems gave **almost nothing** to learn from: 1 correct answer in 160 tries.
- The LiveCodeBench examples it did get were **long** (median 4,709 tokens), and barely shorter than an average correct answer (0.987). On LiveCodeBench answers that finished, LoRA-2 then thought for a median of **3,974** tokens, against **1,066** for LoRA-1.
- LoRA-2 did learn its examples: its training loss fell from about 0.25 to 0.16 ([Figure 8 in FULL-RESULTS](#)).

5.10 From the pilot study to the real test

Table 5.8. LoRA-1 against thinking ON, on different test sets.

Test set	Thinking vs ON	Accuracy vs ON (points)
MBPP+ (pilot: the same kind of problems as its training)	x0.59 [0.46, 0.75]	+15.0 [+6.0, +25.0]
HumanEval+	x0.94 [0.84, 1.05]	+1.2 [-3.7, +6.4]
LiveCodeBench easy	x0.67 [0.55, 0.79]	+17.7 [+3.2, +32.3]
LiveCodeBench medium	x0.96 [0.87, 1.05]	+1.3 [-2.6, +5.1]
All 234 (real test)	x0.87 [0.81, 0.93]	+3.4 [-0.4, +7.7]

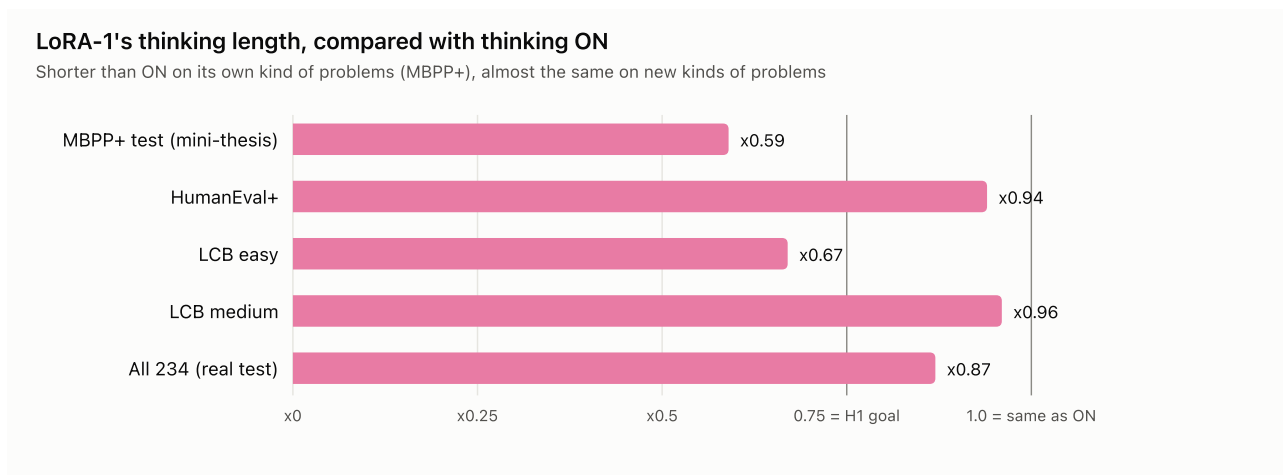


Figure 5.8. LoRA-1's thinking length compared with thinking ON.

On problems like its training data, training worked well (41% less thinking, +15 points). On new kinds of problems, most of the effect disappeared.

5.11 Computer time

- Answering all 234 problems twice took about **38 A100 minutes** for each thinking way and **29 minutes** for thinking OFF.
- The whole run, including making training data, training, and stage D, used about **38 Colab units**.
- OFF writes 4× fewer tokens but was only about 1.3× faster here, because each batch waits for its slowest answer. That is why we measure cost in tokens (Section 4.15).

5.12 Summary of results

Finding	Number
Most accurate way	Thinking limit, 49.8% (+7.7 [+4.3, +11.3] vs ON)
The limit beat the trained model	+4.7 [+0.2, +9.0]
The limit beat thinking OFF	+9.0 [+3.8, +14.3]
Trained model's thinking length	x1.00 of ON (not shorter)
Cheapest way	Thinking OFF: 860 tokens per answer, 40.8% accuracy
Long answers were mostly loops	69.5% of ON's cut-offs, 88.5% of LoRA-2's
More room for ON (16k)	HumanEval+ 53.7% → 59.1%; LiveCodeBench 10.0% → 17.1%
Training helped only on similar problems	x0.59 on MBPP+, x0.94 on HumanEval+

5.13 Size extension: 0.8B and 4B (same family)

The main thesis uses **Qwen3.5-2B**. A teacher may ask: *why only one size?* and *why only limit 1,024?* So we ran the **same 234 problems** on two more models in the same family (DECISIONS #72–#77), and later filled **2B × limit 512** in a small extra run (notebook 16, DECISIONS #81–#82):

Size	Tries	Limits tested	Training
0.8B (lean)	1	512, 1,024	LoRA-1 only
2B (main + fill-in)	2	512 , 1,024 (+ other free ways on main run)	LoRA-1 and LoRA-2 (main run)
4B	2	512, 1,024, 2,048, 4,096	LoRA-1 only

0.8B was lean on purpose: Colab hours were tight after 4B. We dropped Stage C (second try) and limits 2,048 / 4,096.

Captions must say **0.8B = 1 try; 2B / 4B = 2 tries**.

Accuracy on all 234 problems

Way	0.8B (1 try)	2B (2 tries)	4B (2 tries)
Thinking OFF	20.5%	40.8%	69.7%
Thinking ON	7.3%	42.1%	64.3%
Limit 512	17.5%	45.1%	75.4%
Limit 1,024	13.2%	49.8%	76.5%
Limit 2,048	—	—	78.2%
Limit 4,096	—	—	76.7%
LoRA-1	17.9%	45.5%	69.9%

Best free way per size

0.8B → OFF 20.5%
 2B → limit 1,024 49.8% (limit 512 = 45.1%: helps, but 1024 still better)
 4B → limit 2,048 78.2%

Figure idea (for the paper): three bars per size — OFF, best limit (or “n/a”), LoRA-1 — so the eye sees that LoRA never wins.

Cut-off under thinking ON (how often answers hit the wall)

Size	ON accuracy	ON cut-off share
0.8B	7.3%	78%
2B	42.1%	41% (main run)
4B	64.3%	28%

Bigger models finish more often. The tiny 0.8B model almost always hits the wall when thinking is ON.

Raw folders: `results/0.8b/`, `results/4b/`, `results/2b-limit512/`. Join page: `results/SIZE-COMPARISON.md`.

6. Analysis

Chapter 5 showed **what** happened. This chapter explains **why**, compares it with earlier work, and says honestly what could be wrong.

6.1 The answer to the research question

On a small model with a thinking switch, does training on its own shortest correct code answers give a better balance of accuracy and thinking length than the free options?

For Qwen3.5-2B, the answer is no.

	accuracy		tokens per answer
Thinking limit	49.8% ◀ best		2,722
LoRA-2 (trained)	45.1%		3,392 ◀ not shorter than ON
Thinking ON	42.1%		3,446
Thinking OFF	40.8%		860 ◀ cheapest

The trained model did not think shorter (x1.00), and its small accuracy gain could not be proven. The free thinking limit was more accurate than the trained model (+4.7, proven), more accurate than normal thinking (+7.7, proven), and cheaper. Thinking OFF was 4× cheaper than normal thinking, at about the same overall accuracy.

6.2 Why training did not shorten thinking

Reason 1 (the main one): the waste was loops, not long careful thinking

Shortest-correct training is built on one idea of overthinking: *the model finds the answer, then keeps checking*. If that were true, showing the model short correct answers would teach it to stop earlier.

Our data shows a different kind of waste:

What we expected (overthinking)	What we found (looping)
think ... answer found ... check ... check ... check ... </think> answer → finishes, just too long → training CAN shorten it	think ... think ... [repeat] [repeat] [repeat] [repeat] ... ✕ cut off, no answer → never finishes → training on finished answers can't fix it

The evidence:

- Answers that **finished** were already short: median 672 thinking tokens on HumanEval+ (Section 5.5).
- Most answers that **didn't finish** were loops: 69.5% for thinking ON, 88.5% for LoRA-2 (Table 5.4).
- ON, LoRA-1 and LoRA-2 have almost the same length curves up to about 1,000 tokens. They differ in how many answers never finish (Figure 5.4).

- The makers of the model say the same thing. Its official model card warns: *"Qwen3.5-2B is more prone to entering thinking loops compared to other Qwen3.5 models, which may prevent it from terminating generation properly."*

Training only ever shows the model **finished** answers. It never shows how to get **out** of a loop. So it could not remove the main source of waste.

Reason 2: the training signal was weak

- **Little to learn:** for MBPP+, the kept examples were only 17% shorter than an average correct answer (0.829). For LiveCodeBench they were almost exactly average (0.987).
- **Nothing to learn from on medium problems:** 1 correct answer in 160 tries. A model can't learn from problems it can't solve.
- **The wrong lesson on LiveCodeBench:** its few LiveCodeBench examples were long (median 4,709 tokens). LoRA-2 then thought **longer** than LoRA-1 on LiveCodeBench (3,974 against 1,066 tokens on finished answers).

This fits a known finding about small models: *"small models ($\leq 3B$ parameters) do not consistently benefit from long chain-of-thought (CoT) reasoning"* (Li et al., 2025).

Reason 3: the effect did not carry over to new kinds of problems

In the pilot, where training and test problems came from the same benchmark (MBPP+), LoRA-1 cut thinking by 41% and gained 15 points. On HumanEval+, which has similar easy function problems from a different source, the effect almost disappeared (x0.94, +1.2). The training seems to teach **habits for one kind of problem**, not a general "think shorter" skill.

A possible fourth reason (not tested)

SEER (2025) reported that LoRA training was about 7 points less accurate than training the whole model. We only used LoRA. Training the whole model might learn more.

6.3 Why the thinking limit worked

It stops loops by force. When thinking reaches 1,024 tokens, the model must answer.

```
Thinking ON:      think ... [loop] [loop] [loop] ... ✖ cut off at 4,096 → no code → FAIL
Thinking limit:   think ... [loop] ✖ stop at 1,024 → </think> → writes code → often PASS
```

On easy problems, most useful thinking fits inside 1,024 tokens (Figure 5.4: 183 of 328 ON answers on HumanEval+, 55.8%, finished thinking within 1,024 tokens). So when the loop is cut, the model usually already knows enough to write working code. That is why the limit made only 12 problems worse and 38 better.

Its limits:

1. **It moves some of the writing; it doesn't remove all of it.** After the forced stop, the model often keeps reasoning inside its answer (1,879 tokens on average). The real saving is 21% of all tokens, not the 74% the thinking count suggests.
2. **It doesn't help on hard problems.** On LiveCodeBench medium, it solved nothing. The model had not worked out the answer by 1,024 tokens, and it often kept going in the answer part until the overall limit.
3. **On easy problems, part of its win comes from our token limit.** With 16,384 tokens, thinking ON nearly caught up on HumanEval+ (59.1% against 60.4%, Section 5.7). But ON needed up to 16 times more thinking room to get there.

6.4 Thinking OFF is a strong, cheap option

Thinking OFF used 860 tokens per answer, 4× fewer than thinking ON, with about the same overall accuracy (−1.3, not proven). The groups differ:

- On **HumanEval+**, thinking helps: OFF is 7.9 points worse (proven).
- On **LiveCodeBench**, OFF is **better** than normal thinking, on easy (+22.6) and on medium problems (+7.7), because normal thinking so often looped until the limit.

NoThinking (2025) reported that skipping thinking can beat thinking with a small budget, also on code. In our study, the 1,024-token limit beat OFF overall (+9.0, proven), but OFF beat the limit on LiveCodeBench medium (−9.0 for the limit). The studies use different models and budgets, so they don't directly disagree. Both show that "thinking off" must always be part of the comparison.

6.5 How this fits earlier work

Earlier work	What they found	What we add
Munkhbat et al. (2025): shortest-correct training on math	~12% fewer tokens, same accuracy	On a small code model, training did not shorten thinking on new problems
SEER (2025), ASAP (2025): code, 7–8B models	23–40% shorter thinking	Our pilot agrees (41% shorter) only when test problems are like training problems
NoThinking (2025)	Thinking off is a strong option	Confirmed: 4× cheaper, same overall accuracy; better on the harder problems
s1 (2025), Qwen3 report (2025): thinking budgets	Budgets can control how much a model thinks	A budget was the most accurate way for a small model, because it stops loops
Pipis et al. (2025): why models loop	<i>"Larger models tend to loop less"</i>	We measured looping as the main waste in a 2B model, and showed that training on short answers doesn't fix it

Our main new point: **for a small reasoning model, the main waste is thinking that never finishes, not correct thinking that runs long.** Methods that learn only from finished answers cannot address it.

6.6 What could be wrong? (threats to validity)

Is the comparison fair?

-  All ways used the same problems, settings, seeds, token limits and checker. The main LoRA and the hypotheses were fixed before the run.
-  The token limits cut off some useful thinking on HumanEval+. We measured it: +5.5 points for thinking ON with 16,384 tokens (Section 5.7).
-  "Think briefly" was tested with one wording, which clashed with our answer rule.
-  Our loop test is strict and under-counts loops. The true share is probably higher.

Does it hold elsewhere?

-  Main thesis = one model (2B). **Softened:** we also ran **0.8B and 4B** in the same family on the same 234 problems (Section 5.13). The free winner changed with size; LoRA-1 still lost. Other families and harder tasks are still open.
-  Only easy and medium problems. LiveCodeBench medium is at the floor (0–9%) and can't separate the ways.
-  Code only.
-  Size extension: 0.8B used **1 try** (lean plan); 2B/4B used 2 tries.

Is it just luck?

-  234 problems, 2 tries each. Error bars are about ± 4 –6 points over all problems and much wider on the small LiveCodeBench groups (31 and 39 problems). Only fairly large differences can be proven. The main findings (the limit +7.7, LoRA-2 x1.00) are well outside that noise.

Are we measuring the right thing?

-  Accuracy uses the benchmarks' own tests, and the checker passed the official solutions first.
-  LiveCodeBench answers were checked on up to 20 tests per problem, not always all of them.
-  GPU time depends on batching, so we use tokens for cost.

6.7 Size extension: what 0.8B and 4B change

Chapter 5.13 gave the numbers. Here is what they mean.

1. Size still matters most for raw accuracy.

OFF alone goes 20.5% → 40.8% → 69.7% from 0.8B to 4B. Bigger is simply better at code here.

2. The best *free* fix depends on size.

Size	Best free way	Everyday reading
0.8B	Thinking OFF	Too weak to think well — better not to start a long think
2B	Limit 1,024	Thinking helps if you cut loops
4B	Limit 2,048	Same idea; the useful budget moved up

3. Open thinking (ON) is risky when the model loops.

On 0.8B, ON is the *worst* way (7.3%, 78% cut off). On 4B, OFF still beats ON (69.7% vs 64.3%).

So “always leave thinking on” is bad advice for these small code models.

4. LoRA-1 still does not beat the best free way on 0.8B or 4B.

Same story as the main 2B thesis: training on shortest-correct answers is not enough when the free winner is OFF or a hard limit.

5. **Best limit grows with size** where limits help: 1024 on 2B $\rightarrow$ 2048 on 4B. On 2B, **limit 512 = 45.1%** (beats ON 42.1%, loses to 1024 49.8%).

6. Honest caveats.

0.8B used **1 try**; 2B/4B used **2**. Ranking ways *inside* 0.8B is still fair. Cross-size %-point gaps have wider noise on 0.8B.

0.8B did not re-test limits 2,048 / 4,096 (hour budget). 4B already showed 2,048 can win on a larger model. 2B limit 512 came from a separate fill-in run (notebook 16), not from the original thesis Stage A.

This softens one threat from Section 6.6 (“one model only”): we now have **three sizes in one family**. It does **not** remove “code only” or “easy/medium only”.

6.8 Summary of the analysis

```
Small model thinks  $\rightarrow$  often gets STUCK IN A LOOP  $\rightarrow$  never finishes  $\rightarrow$  fails
      |
      Training on short correct answers: never shows how to get unstuck  $\rightarrow$  no change (x1.00)
      Thinking limit:                cuts the loop, forces an answer  $\rightarrow$  +7.7 points (2B)
      Thinking OFF:                  never enters the loop  $\rightarrow$  4 $\times$  cheaper; best on
0.8B

Across sizes (same 234 problems):
  0.8B  $\rightarrow$  OFF wins      · 2B  $\rightarrow$  limit 1024 wins (512 = 45.1%) · 4B  $\rightarrow$  limit 2048 wins
  LoRA-1 never beats that best free way
```

7. Conclusion and Future Work

7.1 The answer

We asked whether training a small reasoning model to think shorter is better than the free options. **On Qwen3.5-2B, for code, it is not.** Training on the model's own shortest correct answers did not shorten its thinking on new problems. The most accurate way was a free one: **stop thinking at 1,024 tokens**. It reached **49.8%**, against **42.1%** for normal thinking (+7.7 points, proven), and it also beat the trained model and thinking off.

We then checked **two more sizes** in the same family (0.8B and 4B) on the **same 234 problems**. The free winner changed with size, but **LoRA-1 never beat the best free way** on any of the three.

The reason is the most important lesson of this thesis:

A small reasoning model doesn't mainly waste tokens by thinking too carefully. It wastes them by getting stuck in loops. Training on short, finished answers can't teach it to get unstuck. A thinking limit simply cuts the loop. On a *very* small model, even a limit is not enough — switching thinking **OFF** can be best.

7.2 Main findings

1. **The thinking limit was the most accurate way on 2B:** +7.7 points over normal thinking, +4.7 over the trained model, +9.0 over thinking off (all proven), with 21% fewer tokens than normal thinking.
2. **Training did not shorten thinking** on the 2B test problems (x1.00). Its accuracy gain (+3.0) could not be proven.
3. **Thinking off was the cheapest on 2B:** 4× fewer tokens than normal thinking, at about the same overall accuracy, and better on the harder LiveCodeBench problems.
4. **Most long answers were loops:** 69.5% of normal thinking's unfinished answers, and 88.5% of the trained model's.
5. **Training only helped on problems like its training data:** 41% less thinking on MBPP+, but almost none on HumanEval+.
6. **More room did not rescue normal thinking:** with 16,384 tokens it gained 5.5 points on HumanEval+ and 7.1 on LiveCodeBench, still below the limit.
7. **Size extension (checked):** best free way = **OFF 20.5%** on 0.8B · **limit 1,024** → **49.8%** on 2B · **limit 2,048** → **78.2%** on 4B. On 2B, **limit 512 = 45.1%** (fill-in): better than ON, worse than 1024. LoRA-1 lost to that free winner on every size. ON cut-off fell as size grew (78% → ~41% → 28%).

7.3 Advice for people who use small reasoning models for code

1. Try THINKING OFF and a THINKING LIMIT first. → free; winner depends on size
2. Pick the limit near your size: ~1k (2B) or ~2k (4B). Tiny models: prefer OFF.
3. Watch for LOOPS: count answers that never finish. → they are the real waste
4. Train to think shorter ONLY if free ways are not enough –
and only if your real problems look like your training problems.

7.4 What the hypothesis taught us

Our hypothesis assumed that the waste was **overthinking**: finished but too-long thinking. That assumption came from studies of larger models. For our small model it was wrong: the waste was **looping**. A rejected hypothesis with a clear, measured reason is a useful result. It tells the next study what to measure **first** (Section 7.5.2).

7.5 Future work

7.5.1 Bigger models: will they loop less, think better, and learn from training?

We already ran **0.8B and 4B** (Section 7.5.4). That answers part of the size question. What is still open:

Still open	Why
9B or larger in the same family	May loop even less; may finally give “long but finished” thinking that LoRA can shorten
Other families (not only Qwen3.5)	One family can share quirks
Harder problems / math	Where careful long thinking may really help

Earlier evidence that larger models may differ:

Evidence	Source	What it suggests
<i>"Qwen3.5-2B is more prone to entering thinking loops compared to other Qwen3.5 models"</i>	Official Qwen3.5-2B model card	The same family's larger models loop less
<i>"Larger models tend to loop less"</i>	Pipis et al. (2025), arXiv:2512.12895	Looping goes down as model size goes up
<i>"small models ($\leq 3B$ parameters) do not consistently benefit from long chain-of-thought (CoT) reasoning"</i>	Li et al. (2025), arXiv:2502.12143	Larger models learn better from reasoning examples
Our own data: finished answers were short; the waste was loops	Chapter 5	If a model loops less, its waste becomes "long but finished" thinking, which is what shortest-correct training can shorten
Our 0.8B / 4B runs	Chapter 5.13	ON cut-off dropped with size (78% → 28%). Best free way moved from OFF → limit 1k → limit 2k. LoRA-1 still lost.

What we expected vs what we saw on 4B:

Expectation	What 4B showed (checked)
Fewer cut-offs than 2B	Yes: ON cut-off 28% (vs ~41% on 2B)
Limit may lose if loops are rare	No: limit 2048 still won (78.2%)
Training may finally win	No: LoRA-1 69.9% < every limit and < OFF

So “bigger → less looping” helped accuracy, but **not** enough for shortest-correct LoRA to beat a free limit on 4B.

How to test a still-bigger model without wasting money. Before full training, run a small check on about 40 problems (4 tries, thinking ON, a large token limit, stop when text repeats). Measure three numbers with rules written **before** looking:

Check	What it tells us	Qwen3.5-2B	Go ahead if
Share of answers that loop	Is the waste loops?	28% of thinking-ON answers (132 of 468)	≤ 10%
Kept length ÷ average correct length	Is there short-but-correct thinking to learn from?	0.83 (MBPP+), 0.99 (LCB)	≤ 0.75
Problems with ≥ 2 correct out of 4	Can it solve the training problems?	56% (MBPP+), 15% (LCB)	≥ 50% in every set

7.5.2 Methods that target loops directly

Our results suggest attacking the loop itself:

- **Stop when the text repeats.** Detect a loop while the model is writing and stop it early. The model card itself recommends *"further tuning the sampling parameters"* and using streaming *"to enable timely detection and interruption of such anomalous generation behaviors."* Our size notebooks already use `--stop-on-repeat` for generation.
- **Change the sampling settings.** The model card says a `presence_penalty` between 0 and 2 can *"reduce endless repetitions"*, with some trade-offs. Pipis et al. (2025) found that a higher temperature reduces looping, though answers stay long. We used the official coding settings (no penalty), so this is untested here.
- **Train on "unstuck" examples.** Instead of only short finished answers, include answers where a loop was cut and the model then answered correctly, like the limit's successful answers. This would teach the model what the limit does by force.
- **Combine the limit with training.** Use the thinking limit on top of a trained model.

7.5.3 Other extensions

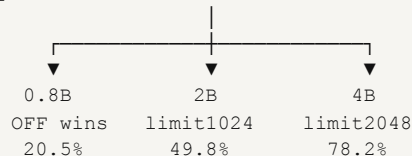
- **Other "think briefly" wordings** that don't clash with the answer rule.
- **More tries per problem** on 0.8B (we used 1 try to save hours).
- **Full fine-tuning** instead of LoRA, given SEER's reported gap between them.
- **Hard problems and math**, where longer, careful thinking may really be needed.
- **Limits we skipped on 0.8B** (2,048 / 4,096), if more GPU hours appear.

7.5.4 Size × limit extension — results (DECISIONS #72–#77)

Status: done. Same 234 problems; notebooks 15a (0.8B) and 15b (4B); folders `results/0.8b/` and `results/4b/`.

Size	Best free way	Score	LoRA-1	Beats LoRA?
0.8B (1 try)	Thinking OFF	20.5%	17.9%	yes
2B (2 tries)	Limit 1,024	49.8%	45.5%	yes
2B limit512 fill-in	Limit 512	45.1%	(same LoRA-1)	limit1024 still wins
4B (2 tries)	Limit 2,048	78.2%	69.9%	yes

Same problems → three sizes



Takeaway for users: try **OFF** and a **short thinking limit** before any training.

Pick a larger limit as the model grows (about 1k → 2k in this family). On a tiny model, prefer OFF.

Full tables: `results/SIZE-COMPARISON.md` • `results/ALL-RESULTS.md`.

7.6 Closing

We set out to teach a small model to stop overthinking, and found that its real problem was getting stuck. For Qwen3.5-2B, the simplest free fix — a limit on thinking — beat training. Across **0.8B, 2B, and 4B**, a free control always beat LoRA-1; only *which* free control won changed with size. Methods that attack loops directly, and tests on still-larger models that finish thinking more often, are the natural next steps.

References

1. Sui, Y. et al. "Stop Overthinking: A Survey on Efficient Reasoning for Large Language Models." *TMLR*, 2025. arXiv:2503.16419.
2. Munkhbat, T. et al. "Self-Training Elicits Concise Reasoning in Large Language Models." *Findings of ACL*, 2025. arXiv:2502.20122.
3. SEER: adaptive chain-of-thought compression for software-engineering tasks. 2025. arXiv:2509.14093.
4. "Pruning the Unsurprising: Efficient Code Reasoning via First-Token Surprisal" (ASAP). 2025. arXiv:2508.05988.
5. "Reasoning Models Can Be Effective Without Thinking" (NoThinking). 2025. arXiv:2504.09858.
6. S3-CoT: self-sampled variable-length chain-of-thought. 2026. arXiv:2602.01982.
7. HRBench: a benchmark of hybrid-reasoning (thinking switch) methods. 2026. arXiv:2605.28398.
8. Muennighoff, N. et al. "s1: Simple test-time scaling." 2025. arXiv:2501.19393.
9. Qwen Team. "Qwen3 Technical Report." 2025. arXiv:2505.09388.
10. Qwen Team. Qwen3.5-2B model card. <https://huggingface.co/Qwen/Qwen3.5-2B> (read 2026-09-25).
11. Pipis, C., Garg, S., Kontonis, V., Shrivastava, V., Krishnamurthy, A., Papailiopoulos, D. "Wait, Wait, Wait... Why Do Reasoning Models Loop?" 2025. arXiv:2512.12895.
12. Li, Y. et al. "Small Models Struggle to Learn from Strong Reasoners." 2025. arXiv:2502.12143.
13. Hu, E. J. et al. "LoRA: Low-Rank Adaptation of Large Language Models." *ICLR*, 2022. arXiv:2106.09685.
14. Liu, J. et al. "Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation" (EvalPlus: HumanEval+, MBPP+). *NeurIPS*, 2023. arXiv:2305.01210.
15. Jain, N. et al. "LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code." 2024. arXiv:2403.07974.
16. Austin, J. et al. "Program Synthesis with Large Language Models" (MBPP). 2021. arXiv:2108.07732.
17. Unsloth. Fast LoRA fine-tuning library and Qwen3.5 guide. <https://unsloth.ai>.

Where each paper is discussed: 1–6 and 13–16 in Chapter 2; 5, 8, 9 in Sections 2.8 and 6.4–6.5; 10–12 in Sections 2.5, 2.9, 6.2 and 7.5. Notes on every paper we read are in [PAPERS.md](#).

Appendices

Appendix A. Every problem, every number, every chart

The full results page lists **every** test problem (all 234, with how many of its 2 tries each way solved) and **every** training problem (all 280, with how many of its 4 tries were correct and whether it was kept):

- [results/full-results/FULL-RESULTS.md](#), Appendix A (test problems) and Appendix B (training problems)
- The same as spreadsheets: [per-problem.csv](#), [training-problems.csv](#), [per-way-per-group.csv](#), [gpu-time.csv](#)
- All 10 charts: [results/full-results/figures/](#)

Appendix B. How to repeat this work

Re-check every number (no GPU needed, about one minute):

```
node scripts/make_full_results.js      → rebuilds FULL-RESULTS.md, the charts and the tables
node scripts/check_thesis_run.js results/2026-09-24-thesis-run → loops, stage D, training data
```

Re-run the whole experiment (needs a Colab A100 and about 40 units):

1. Open `notebooks/14_thesis_run.ipynb` in Google Colab. Choose **Runtime** → **Change runtime type** → **A100 GPU**.
2. Put the pilot LoRA (LoRA-1) in `MyDrive/stop-overthinking/results/mini/lora/lora100` (made by `notebooks/13_mini_thesis.ipynb`).
3. **Runtime** → **Run all**. Type the number of Colab units left when asked.
4. The notebook runs every stage, skips what is already done after a disconnect, and prints the results tables at the end.

Key scripts:

Script	What it does
<code>scripts/gen_colab.py</code>	Makes the model answer, for every way (including the thinking limit and the LoRAs)
<code>scripts/prompts.py</code>	Builds the question and splits an answer into thinking and answer part
<code>scripts/grade_plus.py</code> , <code>scripts/grade_lcb.py</code>	Run the benchmarks' tests in a separate process
<code>scripts/overlap_check.py</code>	Removes training problems that are too close to a test problem
<code>scripts/make_train_set.py</code>	Keeps the shortest correct answer (at least half the median)
<code>scripts/train_lora.py</code>	Trains a LoRA
<code>scripts/compare_thesis.py</code>	Accuracy, token ratios, error bars, hypotheses

Appendix C. How the project developed

Date (2026)	Step
13 Sept	Topic, gap and plan written; proposal written
19–20 Sept	First model calls. Gemma-4-E4B too slow on free GPUs; pilot on 30 HumanEval problems
20 Sept	Moved to Google Colab and Qwen3.5-2B; test set fixed at 234 problems
22 Sept	Pilot study (the "mini-thesis") on MBPP+: training shortened thinking by 41% and gained 15 points
22 Sept	Rules for the main run fixed before any result (token limits, main LoRA, budget)
21–24 Sept	Main run on a Colab A100
25 Sept	Raw answers checked (loops found); results and thesis written

Every choice and its reason is logged, with dates, in [DECISIONS.md](#) (69+ entries).

Appendix D. Words used in this thesis

Word	Meaning
Accuracy	Answers that passed all tests ÷ all answers
Benchmark	A fixed set of problems with a fair way to check the answers
Bootstrap	A way to get error bars: re-draw the problems at random 2,000 times and see how much the result moves
Cut off	The answer reached the token limit before it finished; it counts as a fail
Epoch	One pass over all the training examples
Error bar (95%)	The range the true difference very likely lies in; if it doesn't include 0, the difference is proven
Fine-tuning / training	Nudging the model's numbers with examples, so it writes more like them
GPU	The graphics card that runs the model
LoRA	A small add-on trained on top of the model; the model itself stays unchanged
Loop	The model repeats the same lines until it runs out of room
Median	The middle value when the numbers are sorted
Overthinking	Thinking far longer than needed, but still finishing
Parameter	One of the billions of numbers that hold what the model learned
Percentage point	The simple difference between two percentages: 49.8% – 42.1% = 7.7 points
Reasoning model	A model that writes a thinking part before its answer
Seed	A fixed starting number for random choices, so results can be repeated exactly
Test case	One input and the output the code must give for it
Thinking limit (budget)	Stop the thinking after a fixed number of tokens and make the model answer
Thinking ratio	A way's thinking tokens ÷ normal thinking's, on the same problems (x1.00 = the same)
Thinking switch	A setting that turns the thinking part on or off
Token	A small piece of text, about ¾ of a word; the unit of cost

More words are explained in [GLOSSARY.md](#).

Printed appendices

These two lists are copied from `results/full-results/FULL-RESULTS.md` so the printed thesis is complete.

Appendix E. Every test problem (234)

Each cell = how many of the 2 tries passed. **ON 16k** = the try-1 re-run with 16,384 tokens (only for cut-off answers).

#	Problem	Set	Level	Thinking ON	Thinking OFF	Think briefly	Limit 1,024	LoRA-1	LoRA-2	ON 16k
1	HumanEval/0	HE+	easy	1	0	0	2	2	2	
2	HumanEval/1	HE+	easy	0	0	0	0	0	0	✗
3	HumanEval/2	HE+	easy	2	2	0	2	2	2	
4	HumanEval/3	HE+	easy	2	2	0	2	2	2	
5	HumanEval/4	HE+	easy	1	2	0	2	2	2	✓
6	HumanEval/5	HE+	easy	2	0	0	2	2	2	
7	HumanEval/6	HE+	easy	0	0	0	0	2	0	✓
8	HumanEval/7	HE+	easy	2	2	1	2	2	2	
9	HumanEval/8	HE+	easy	2	2	0	2	2	2	
10	HumanEval/9	HE+	easy	1	2	0	1	2	2	
11	HumanEval/10	HE+	easy	0	0	0	0	0	0	✗
12	HumanEval/11	HE+	easy	2	2	0	2	2	2	
13	HumanEval/12	HE+	easy	2	2	0	2	2	2	
14	HumanEval/13	HE+	easy	2	1	0	2	2	2	
15	HumanEval/14	HE+	easy	2	2	0	2	2	2	
16	HumanEval/15	HE+	easy	2	2	2	2	2	2	
17	HumanEval/16	HE+	easy	2	2	0	2	2	1	
18	HumanEval/17	HE+	easy	2	0	0	2	2	1	
19	HumanEval/18	HE+	easy	2	2	0	2	2	1	
20	HumanEval/19	HE+	easy	1	1	0	1	0	1	
21	HumanEval/20	HE+	easy	1	0	0	1	0	0	
22	HumanEval/21	HE+	easy	2	1	0	2	2	2	
23	HumanEval/22	HE+	easy	0	0	0	0	1	0	
24	HumanEval/23	HE+	easy	2	2	1	2	2	2	
25	HumanEval/24	HE+	easy	1	1	0	2	2	1	✓
26	HumanEval/25	HE+	easy	2	2	0	2	2	1	
27	HumanEval/26	HE+	easy	0	0	0	0	0	0	
28	HumanEval/27	HE+	easy	1	0	0	0	2	2	✓
29	HumanEval/28	HE+	easy	2	2	1	2	2	2	
30	HumanEval/29	HE+	easy	2	2	0	2	2	2	
31	HumanEval/30	HE+	easy	2	2	0	2	2	2	
32	HumanEval/31	HE+	easy	2	2	0	2	2	2	
33	HumanEval/32	HE+	easy	0	0	0	0	0	0	✗
34	HumanEval/33	HE+	easy	1	1	0	1	1	0	
35	HumanEval/34	HE+	easy	2	2	0	2	2	2	

#	Problem	Set	Level	Thinking ON	Thinking OFF	Think briefly	Limit 1,024	LoRA-1	LoRA-2	ON 16k
36	HumanEval/35	HE+	easy	2	2	1	2	2	2	
37	HumanEval/36	HE+	easy	0	1	0	2	0	1	✗
38	HumanEval/37	HE+	easy	0	0	0	2	0	1	✗
39	HumanEval/38	HE+	easy	0	0	0	0	0	1	✗
40	HumanEval/39	HE+	easy	1	0	0	0	0	0	✗
41	HumanEval/40	HE+	easy	0	1	0	2	0	1	✗
42	HumanEval/41	HE+	easy	0	2	0	0	1	1	
43	HumanEval/42	HE+	easy	2	2	1	2	2	2	
44	HumanEval/43	HE+	easy	1	2	0	1	1	2	
45	HumanEval/44	HE+	easy	0	2	0	0	0	1	
46	HumanEval/45	HE+	easy	2	2	0	2	2	2	
47	HumanEval/46	HE+	easy	1	2	0	1	1	2	
48	HumanEval/47	HE+	easy	2	2	0	2	1	2	
49	HumanEval/48	HE+	easy	2	2	0	2	1	2	
50	HumanEval/49	HE+	easy	2	2	0	2	2	2	
51	HumanEval/50	HE+	easy	1	1	0	1	1	2	
52	HumanEval/51	HE+	easy	2	2	0	2	2	2	
53	HumanEval/52	HE+	easy	2	2	0	2	2	2	
54	HumanEval/53	HE+	easy	2	2	0	2	2	2	
55	HumanEval/54	HE+	easy	0	0	0	0	0	0	
56	HumanEval/55	HE+	easy	2	2	0	2	2	2	
57	HumanEval/56	HE+	easy	1	2	1	2	2	2	
58	HumanEval/57	HE+	easy	1	1	0	1	1	1	
59	HumanEval/58	HE+	easy	2	2	0	2	2	2	
60	HumanEval/59	HE+	easy	1	1	0	1	2	1	
61	HumanEval/60	HE+	easy	2	2	1	2	2	2	
62	HumanEval/61	HE+	easy	2	2	0	2	2	2	
63	HumanEval/62	HE+	easy	0	1	0	1	0	0	✗
64	HumanEval/63	HE+	easy	2	2	0	2	0	2	
65	HumanEval/64	HE+	easy	0	0	0	0	0	0	
66	HumanEval/65	HE+	easy	0	1	0	1	0	1	✗
67	HumanEval/66	HE+	easy	2	2	0	2	2	2	
68	HumanEval/67	HE+	easy	2	0	0	2	1	1	
69	HumanEval/68	HE+	easy	1	2	0	1	1	0	
70	HumanEval/69	HE+	easy	2	2	0	2	2	1	
71	HumanEval/70	HE+	easy	0	0	0	0	0	0	
72	HumanEval/71	HE+	easy	2	0	0	2	2	2	
73	HumanEval/72	HE+	easy	2	2	0	2	2	2	
74	HumanEval/73	HE+	easy	2	0	0	2	2	2	
75	HumanEval/74	HE+	easy	2	0	0	2	2	2	
76	HumanEval/75	HE+	easy	1	0	0	0	0	2	
77	HumanEval/76	HE+	easy	0	0	0	0	0	0	
78	HumanEval/77	HE+	easy	0	0	0	0	0	0	✗
79	HumanEval/78	HE+	easy	1	2	0	2	2	2	✓
80	HumanEval/79	HE+	easy	1	2	0	1	2	2	
81	HumanEval/80	HE+	easy	2	2	1	2	2	2	
82	HumanEval/81	HE+	easy	1	0	0	1	0	0	

#	Problem	Set	Level	Thinking ON	Thinking OFF	Think briefly	Limit 1,024	LoRA-1	LoRA-2	ON 16k
83	HumanEval/82	HE+	easy	2	2	1	2	2	2	
84	HumanEval/83	HE+	easy	0	0	0	1	0	0	✗
85	HumanEval/84	HE+	easy	1	0	0	1	1	1	
86	HumanEval/85	HE+	easy	2	1	0	2	2	2	
87	HumanEval/86	HE+	easy	0	0	0	0	0	0	✗
88	HumanEval/87	HE+	easy	1	0	0	2	1	1	
89	HumanEval/88	HE+	easy	1	2	0	2	0	0	
90	HumanEval/89	HE+	easy	1	1	0	0	1	1	
91	HumanEval/90	HE+	easy	0	1	0	0	1	1	
92	HumanEval/91	HE+	easy	0	0	0	0	0	0	✗
93	HumanEval/92	HE+	easy	2	2	0	2	1	2	
94	HumanEval/93	HE+	easy	0	0	0	0	0	0	
95	HumanEval/94	HE+	easy	1	1	0	2	1	2	✓
96	HumanEval/95	HE+	easy	1	2	0	1	2	0	
97	HumanEval/96	HE+	easy	0	2	0	0	1	1	
98	HumanEval/97	HE+	easy	0	0	0	0	1	0	
99	HumanEval/98	HE+	easy	2	2	0	2	1	1	
100	HumanEval/99	HE+	easy	0	1	0	0	2	0	✓
101	HumanEval/100	HE+	easy	2	0	0	2	1	0	✗
102	HumanEval/101	HE+	easy	0	0	0	1	0	1	
103	HumanEval/102	HE+	easy	2	1	0	2	2	2	
104	HumanEval/103	HE+	easy	0	0	0	0	1	0	
105	HumanEval/104	HE+	easy	1	1	0	2	2	2	
106	HumanEval/105	HE+	easy	2	2	0	2	2	2	
107	HumanEval/106	HE+	easy	1	0	0	1	1	2	
108	HumanEval/107	HE+	easy	1	0	0	0	1	1	
109	HumanEval/108	HE+	easy	0	0	0	0	0	0	
110	HumanEval/109	HE+	easy	2	1	0	1	1	0	
111	HumanEval/110	HE+	easy	2	1	0	1	1	0	
112	HumanEval/111	HE+	easy	1	2	0	1	0	2	
113	HumanEval/112	HE+	easy	1	1	0	1	0	1	
114	HumanEval/113	HE+	easy	0	0	0	0	1	0	
115	HumanEval/114	HE+	easy	0	0	0	0	1	1	
116	HumanEval/115	HE+	easy	0	0	0	0	1	1	
117	HumanEval/116	HE+	easy	1	2	0	2	0	0	
118	HumanEval/117	HE+	easy	2	0	0	2	0	0	
119	HumanEval/118	HE+	easy	0	0	0	0	0	0	✗
120	HumanEval/119	HE+	easy	1	1	0	1	1	1	✗
121	HumanEval/120	HE+	easy	0	1	0	2	1	1	✗
122	HumanEval/121	HE+	easy	2	1	1	2	1	2	
123	HumanEval/122	HE+	easy	0	0	0	0	0	0	
124	HumanEval/123	HE+	easy	2	0	0	2	1	0	
125	HumanEval/124	HE+	easy	0	0	0	0	0	0	
126	HumanEval/125	HE+	easy	0	0	0	0	0	0	
127	HumanEval/126	HE+	easy	0	1	0	0	0	0	✓
128	HumanEval/127	HE+	easy	2	0	0	2	1	2	
129	HumanEval/128	HE+	easy	2	0	0	1	1	2	

#	Problem	Set	Level	Thinking ON	Thinking OFF	Think briefly	Limit 1,024	LoRA-1	LoRA-2	ON 16k
130	HumanEval/129	HE+	easy	0	0	0	0	0	0	✗
131	HumanEval/130	HE+	easy	0	0	0	0	0	0	
132	HumanEval/131	HE+	easy	0	0	0	0	0	0	
133	HumanEval/132	HE+	easy	0	0	0	0	0	0	✗
134	HumanEval/133	HE+	easy	2	0	0	2	2	2	
135	HumanEval/134	HE+	easy	0	0	0	0	0	0	✗
136	HumanEval/135	HE+	easy	1	1	0	1	1	0	
137	HumanEval/136	HE+	easy	1	0	0	1	2	2	
138	HumanEval/137	HE+	easy	0	0	0	0	2	1	✗
139	HumanEval/138	HE+	easy	2	0	0	2	2	2	
140	HumanEval/139	HE+	easy	2	0	0	2	2	2	
141	HumanEval/140	HE+	easy	0	0	0	0	0	0	
142	HumanEval/141	HE+	easy	2	0	0	1	1	1	
143	HumanEval/142	HE+	easy	2	0	0	2	1	2	
144	HumanEval/143	HE+	easy	2	0	0	2	2	2	
145	HumanEval/144	HE+	easy	1	1	0	2	2	2	
146	HumanEval/145	HE+	easy	0	0	0	0	0	0	
147	HumanEval/146	HE+	easy	2	2	0	2	1	1	
148	HumanEval/147	HE+	easy	0	0	0	0	1	0	✗
149	HumanEval/148	HE+	easy	0	1	0	0	1	2	
150	HumanEval/149	HE+	easy	1	1	0	2	2	2	
151	HumanEval/150	HE+	easy	2	2	0	2	2	2	
152	HumanEval/151	HE+	easy	2	2	0	2	2	2	
153	HumanEval/152	HE+	easy	2	2	0	2	2	2	
154	HumanEval/153	HE+	easy	1	2	0	2	2	2	
155	HumanEval/154	HE+	easy	0	0	0	1	0	0	✗
156	HumanEval/155	HE+	easy	2	2	0	2	2	1	
157	HumanEval/156	HE+	easy	0	0	0	0	0	0	
158	HumanEval/157	HE+	easy	1	2	0	1	2	0	✓
159	HumanEval/158	HE+	easy	1	2	0	1	1	2	
160	HumanEval/159	HE+	easy	1	0	0	0	0	1	✗
161	HumanEval/160	HE+	easy	0	0	0	1	0	0	✗
162	HumanEval/161	HE+	easy	2	0	0	2	2	0	
163	HumanEval/162	HE+	easy	2	1	0	2	1	2	
164	HumanEval/163	HE+	easy	0	0	0	0	0	0	✗
165	lcb/arc192_a	LCB	medium	0	0	0	0	0	0	✗
166	lcb/arc194_a	LCB	medium	0	0	0	0	0	0	✗
167	lcb/arc195_a	LCB	medium	0	0	0	0	0	0	✗
168	lcb/abc391_a	LCB	easy	2	2	0	2	1	2	
169	lcb/abc391_b	LCB	easy	0	0	0	1	0	0	✗
170	lcb/abc391_d	LCB	medium	0	0	0	0	0	0	✗
171	lcb/abc392_a	LCB	easy	0	2	2	2	2	0	✗
172	lcb/abc392_b	LCB	easy	0	2	1	1	1	2	✓
173	lcb/abc392_c	LCB	medium	0	0	0	0	0	0	✗
174	lcb/abc392_d	LCB	medium	0	0	0	0	0	0	✗
175	lcb/abc393_a	LCB	easy	0	1	1	2	2	2	✓
176	lcb/abc393_b	LCB	easy	0	2	0	1	0	0	✗

#	Problem	Set	Level	Thinking ON	Thinking OFF	Think briefly	Limit 1,024	LoRA-1	LoRA-2	ON 16k
177	lcb/abc394_a	LCB	easy	2	2	2	2	2	1	
178	lcb/abc394_b	LCB	easy	1	2	1	2	2	2	
179	lcb/abc394_c	LCB	medium	0	0	0	0	0	0	✗
180	lcb/abc394_d	LCB	medium	0	0	0	0	0	0	✗
181	lcb/abc395_a	LCB	easy	1	2	2	2	2	2	
182	lcb/abc395_b	LCB	easy	0	0	0	0	0	0	✗
183	lcb/abc395_c	LCB	medium	0	0	0	0	0	0	✗
184	lcb/abc396_a	LCB	easy	2	2	2	2	2	2	
185	lcb/abc396_b	LCB	easy	0	1	0	0	0	0	✗
186	lcb/abc396_c	LCB	medium	0	0	0	0	0	0	✗
187	lcb/abc396_d	LCB	medium	0	2	0	0	0	0	✗
188	lcb/abc397_a	LCB	easy	2	2	1	2	2	2	
189	lcb/abc397_b	LCB	medium	0	0	0	0	0	0	✗
190	lcb/abc397_c	LCB	medium	0	1	0	0	0	0	✗
191	lcb/abc398_a	LCB	easy	0	0	0	0	0	0	✗
192	lcb/abc398_b	LCB	medium	0	1	0	0	0	1	✗
193	lcb/abc398_c	LCB	medium	0	0	0	0	0	0	✗
194	lcb/abc399_a	LCB	easy	1	2	1	1	2	2	✓
195	lcb/abc399_b	LCB	easy	0	0	0	0	0	0	✗
196	lcb/abc399_c	LCB	medium	0	1	0	0	1	0	✗
197	lcb/abc399_d	LCB	medium	0	0	0	0	0	0	✗
198	lcb/abc400_a	LCB	easy	1	1	2	2	2	1	✓
199	lcb/abc400_b	LCB	easy	1	1	0	2	2	1	✗
200	lcb/abc400_c	LCB	medium	0	1	0	0	0	0	✗
201	lcb/abc400_d	LCB	medium	0	0	0	0	0	0	✗
202	lcb/3705	LCB	easy	0	0	0	0	0	0	✗
203	lcb/3709	LCB	easy	0	1	0	2	1	1	✗
204	lcb/3722	LCB	medium	0	0	0	0	0	0	✗
205	lcb/3723	LCB	easy	0	0	0	1	0	0	✗
206	lcb/3736	LCB	easy	0	0	1	2	1	0	✗
207	lcb/3743	LCB	medium	0	0	0	0	0	0	✗
208	lcb/3748	LCB	medium	0	0	0	0	0	0	✗
209	lcb/3750	LCB	medium	0	0	0	0	0	0	✗
210	lcb/3753	LCB	easy	0	0	0	2	0	0	✗
211	lcb/3754	LCB	medium	0	0	0	0	0	0	✗
212	lcb/3759	LCB	medium	0	0	0	0	0	0	✗
213	lcb/3760	LCB	medium	0	0	0	0	0	0	
214	lcb/3763	LCB	medium	0	0	0	0	0	0	✗
215	lcb/3764	LCB	medium	1	0	0	0	0	0	
216	lcb/3768	LCB	easy	0	2	0	1	2	1	✗
217	lcb/3771	LCB	medium	0	0	0	0	0	0	✗
218	lcb/3773	LCB	easy	0	0	0	0	0	0	✗
219	lcb/3776	LCB	medium	0	0	0	0	0	0	✗
220	lcb/3778	LCB	easy	0	1	1	0	0	2	✗
221	lcb/3779	LCB	medium	0	0	0	0	0	0	✗
222	lcb/3785	LCB	medium	0	0	0	0	0	0	✗
223	lcb/3786	LCB	medium	0	0	0	0	0	0	✗

#	Problem	Set	Level	Thinking ON	Thinking OFF	Think briefly	Limit 1,024	LoRA-1	LoRA-2	ON 16k
224	lcb/3788	LCB	easy	0	0	0	0	0	0	
225	lcb/3791	LCB	medium	0	0	0	0	0	0	✗
226	lcb/3793	LCB	medium	0	0	0	0	0	0	✗
227	lcb/3794	LCB	medium	0	0	0	0	0	0	✗
228	lcb/3795	LCB	medium	0	0	0	0	0	0	✗
229	lcb/3799	LCB	easy	0	0	0	0	0	0	✗
230	lcb/3805	LCB	medium	0	0	0	0	0	0	✗
231	lcb/3809	LCB	medium	0	1	0	0	1	1	✗
232	lcb/3811	LCB	easy	2	0	1	1	0	1	
233	lcb/3817	LCB	easy	0	1	1	2	0	0	✓
234	lcb/3832	LCB	easy	0	0	0	0	0	0	✗

Appendix F. Every training problem (280)

Correct = how many of the 4 tries passed. Kept = became a training example (the shortest correct answer).

#	Problem	Pool	Level	Correct (of 4)	Kept	Kept thinking tokens	Question starts with
1	lcb/abc301_d	LCB old	medium	0			
2	lcb/abc302_c	LCB old	medium	0			
3	lcb/abc302_d	LCB old	medium	0			
4	lcb/abc303_c	LCB old	medium	0			
5	lcb/abc303_d	LCB old	medium	0			
6	lcb/abc307_a	LCB old	easy	2	✓	4862	Takahashi has recorded the number of steps he walked for N weeks. He w
7	lcb/abc308_a	LCB old	easy	1	✓	7198	Given eight integers S_1,S_2,\dots, and S_8, print Yes if they satisfy
8	lcb/abc308_b	LCB old	easy	0			
9	lcb/abc309_c	LCB old	medium	0			
10	lcb/abc309_d	LCB old	medium	0			
11	lcb/abc311_d	LCB old	medium	0			
12	lcb/abc312_a	LCB old	easy	4	✓	3516	Given a length-3 string S consisting of uppercase English letters, pri
13	lcb/abc312_b	LCB old	easy	0			
14	lcb/abc313_a	LCB old	easy	1	✓	6224	There are N people numbered 1 through N. Each person has a integer sco
15	lcb/abc314_a	LCB old	easy	1	✓	7596	The number pi to the 100-th decimal place is 3.14159265358979323846264
16	lcb/abc314_c	LCB old	medium	0			
17	lcb/abc315_a	LCB old	easy	3	✓	2233	You are given a string S consisting of lowercase English letters. Remo
18	lcb/abc318_d	LCB old	medium	0			

#	Problem	Pool	Level	Correct (of 4)	Kept	Kept thinking tokens	Question starts with
19	lcb/abc319_b	LCB old	easy	1	✓	5837	You are given a positive integer N. Print a string of length (N+1), s_
20	lcb/abc320_a	LCB old	easy	2	✓	5094	You are given positive integers A and B. Print the value $A^B + B^A$. Inpu
21	lcb/abc321_c	LCB old	medium	0			
22	lcb/abc322_a	LCB old	easy	0			
23	lcb/abc322_c	LCB old	medium	0			
24	lcb/abc325_b	LCB old	medium	0			
25	lcb/abc326_b	LCB old	easy	1	✓	1962	A 326-like number is a three-digit positive integer where the product
26	lcb/abc327_b	LCB old	easy	0			
27	lcb/abc329_a	LCB old	easy	2	✓	4613	You are given a string S consisting of uppercase English letters. Sepa
28	lcb/abc329_b	LCB old	easy	3	✓	5548	You are given N integers $A_1, A_2, \dots, A_N$. Find the largest among
29	lcb/abc331_c	LCB old	medium	0			
30	lcb/abc333_a	LCB old	easy	1	✓	4709	You are given an integer N between 1 and 9, inclusive, as input. Conca
31	lcb/abc333_b	LCB old	easy	0			
32	lcb/abc334_a	LCB old	easy	4	✓	1310	Takahashi, a young baseball enthusiast, has been a very good boy this
33	lcb/abc334_c	LCB old	medium	0			
34	lcb/abc335_b	LCB old	easy	0			
35	lcb/abc336_b	LCB old	easy	2	✓	6833	For a positive integer X, let $\text{ctz}(X)$ be the (maximal) number of
36	lcb/abc336_c	LCB old	medium	0			
37	lcb/abc337_c	LCB old	medium	0			
38	lcb/abc339_c	LCB old	medium	0			
39	lcb/abc339_d	LCB old	medium	0			
40	lcb/abc340_a	LCB old	easy	3	✓	7006	Print an arithmetic sequence with first term A, last term B, and commo
41	lcb/abc340_c	LCB old	medium	0			
42	lcb/abc341_b	LCB old	easy	0			
43	lcb/abc341_c	LCB old	medium	0			
44	lcb/abc342_a	LCB old	easy	0			
45	lcb/abc342_b	LCB old	easy	0			
46	lcb/abc342_c	LCB old	medium	0			
47	lcb/1873_D	LCB old	easy	0			
48	lcb/2730	LCB old	medium	0			

#	Problem	Pool	Level	Correct (of 4)	Kept	Kept thinking tokens	Question starts with
49	1cb/2791	LCB old	easy	0			
50	1cb/2792	LCB old	medium	0			
51	1cb/2811	LCB old	medium	0			
52	1cb/2844	LCB old	easy	1	✓	5772	You are given a 1-indexed integer array nums of length n. An element n
53	1cb/2850	LCB old	medium	0			
54	1cb/2854	LCB old	medium	0			
55	1cb/2857	LCB old	easy	0			
56	1cb/2866	LCB old	easy	0			
57	1cb/2868	LCB old	medium	0			
58	1cb/2869	LCB old	medium	0			
59	1cb/2872	LCB old	medium	0			
60	1cb/2886	LCB old	easy	1	✓	1253	Your laptop keyboard is faulty, and whenever you type a character 'i'
61	1cb/2998	LCB old	easy	1	✓	1117	You are given two positive integers low and high. An integer x consist
62	1cb/3018	LCB old	medium	0			
63	1cb/3033	LCB old	medium	0			
64	1cb/3080	LCB old	medium	0			
65	1cb/3093	LCB old	easy	2	✓	688	You are given a 0-indexed integer array nums and an integer k. Return
66	1cb/3164	LCB old	easy	0			
67	1cb/3176	LCB old	easy	1	✓	755	You are given a 0-indexed array nums of integers. A triplet of indices
68	1cb/3190	LCB old	medium	0			
69	1cb/3194	LCB old	easy	2	✓	7574	You are given a 0-indexed array of strings words and a character x. Re
70	1cb/3213	LCB old	medium	0			
71	1cb/3225	LCB old	medium	0			
72	1cb/3226	LCB old	easy	0			
73	1cb/3230	LCB old	medium	0			
74	1cb/3249	LCB old	medium	0			
75	1cb/3251	LCB old	easy	3	✓	726	You are given a 2D 0-indexed integer array dimensions. For all indices
76	1cb/3263	LCB old	easy	0			
77	1cb/3269	LCB old	medium	1	✓	822	You are given a 0-indexed integer array nums of size n, and a 0-indexe
78	1cb/3292	LCB old	medium	0			

#	Problem	Pool	Level	Correct (of 4)	Kept	Kept thinking tokens	Question starts with
79	lcb/3312	LCB old	easy	1	✓	1039	You are given a 0-indexed string s typed by a user. Changing a key is
80	lcb/3331	LCB old	easy	0			
81	Mbpp/7	MBPP+	easy	0			
82	Mbpp/8	MBPP+	easy	4	✓	216	Write a function to find squares of individual elements in a list. You
83	Mbpp/9	MBPP+	easy	0			
84	Mbpp/12	MBPP+	easy	4	✓	527	Write a function to sort a given matrix in ascending order according t
85	Mbpp/14	MBPP+	easy	2	✓	414	Write a python function to find the volume of a triangular prism. Your
86	Mbpp/16	MBPP+	easy	0			
87	Mbpp/17	MBPP+	easy	3	✓	113	Write a function that returns the perimeter of a square given its side
88	Mbpp/56	MBPP+	easy	3	✓	501	Write a python function to check if a given number is one less than tw
89	Mbpp/59	MBPP+	easy	1	✓	581	Write a function to find the nth octagonal number. Your code must pass
90	Mbpp/63	MBPP+	easy	0			
91	Mbpp/65	MBPP+	easy	1	✓	628	Write a function to flatten a list and sum all of its elements. Your c
92	Mbpp/66	MBPP+	easy	4	✓	1935	Write a python function to count the number of positive numbers in a l
93	Mbpp/67	MBPP+	easy	1	✓	1100	Write a function to find the number of ways to partition a set of Bell
94	Mbpp/68	MBPP+	easy	1	✓	1861	Write a python function to check whether the given array is monotonic
95	Mbpp/74	MBPP+	easy	0			
96	Mbpp/75	MBPP+	easy	3	✓	377	Write a function to find tuples which have all elements divisible by k
97	Mbpp/79	MBPP+	easy	2	✓	415	Write a python function to check whether the length of the word is odd
98	Mbpp/82	MBPP+	easy	4	✓	296	Write a function to find the volume of a sphere. Your code must pass t
99	Mbpp/84	MBPP+	easy	0			
100	Mbpp/85	MBPP+	easy	2	✓	250	Write a function to find the surface area of a sphere. Your code must
101	Mbpp/87	MBPP+	easy	2	✓	1052	Write a function to merge three dictionaries into a single dictionary.
102	Mbpp/88	MBPP+	easy	4	✓	435	Write a function to get the frequency of all the elements in a list, r
103	Mbpp/90	MBPP+	easy	3	✓	264	Write a python function to find the length of the longest word. Your c
104	Mbpp/91	MBPP+	easy	1	✓	384	Write a function to check if a string is present as a substring in a g
105	Mbpp/92	MBPP+	easy	0			
106	Mbpp/93	MBPP+	easy	2	✓	172	Write a function to calculate the value of 'a' to the power 'b'. Your
107	Mbpp/94	MBPP+	easy	3	✓	670	Given a list of tuples, write a function that returns the first value
108	Mbpp/95	MBPP+	easy	3	✓	456	Write a python function to find the length of the smallest list in a l
109	Mbpp/96	MBPP+	easy	4	✓	636	Write a python function to find the number of divisors of a given inte
110	Mbpp/98	MBPP+	easy	1	✓	754	Write a function to multiply all the numbers in a list and divide with
111	Mbpp/99	MBPP+	easy	0			

#	Problem	Pool	Level	Correct (of 4)	Kept	Kept thinking tokens	Question starts with
112	Mbpps/100	MBPP+	easy	0			
113	Mbpps/101	MBPP+	easy	3	✓	225	Write a function to find the kth element in the given array using 1-based
114	Mbpps/106	MBPP+	easy	0			
115	Mbpps/111	MBPP+	easy	1	✓	635	Write a function to find the common elements in given nested lists. Yo
116	Mbpps/116	MBPP+	easy	1	✓	412	Write a function to convert a given tuple of positive integers into a
117	Mbpps/118	MBPP+	easy	3	✓	369	Write a function to convert a string to a list of strings split on the
118	Mbpps/119	MBPP+	easy	0			
119	Mbpps/123	MBPP+	easy	0			
120	Mbpps/126	MBPP+	easy	0			
121	Mbpps/130	MBPP+	easy	3	✓	1106	Write a function to find the item with maximum frequency in a given li
122	Mbpps/133	MBPP+	easy	4	✓	1516	Write a function to calculate the sum of the negative numbers of a giv
123	Mbpps/135	MBPP+	easy	2	✓	3926	Write a function to find the nth hexagonal number. Your code must pass
124	Mbpps/137	MBPP+	easy	0			
125	Mbpps/140	MBPP+	easy	2	✓	588	Write a function to flatten the list of lists into a single set of num
126	Mbpps/142	MBPP+	easy	0			
127	Mbpps/145	MBPP+	easy	2	✓	226	Write a python function to find the maximum difference between any two
128	Mbpps/161	MBPP+	easy	4	✓	360	Write a function to remove all elements from a given list present in a
129	Mbpps/165	MBPP+	easy	1	✓	2223	Write a function to count the number of characters in a string that oc
130	Mbpps/170	MBPP+	easy	1	✓	523	Write a function to find the sum of numbers in a list within a range s
131	Mbpps/171	MBPP+	easy	3	✓	2067	Write a function to find the perimeter of a regular pentagon from the
132	Mbpps/222	MBPP+	easy	2	✓	597	Write a function to check if all the elements in tuple have same data
133	Mbpps/224	MBPP+	easy	4	✓	237	Write a python function to count the number of set bits (binary digits
134	Mbpps/227	MBPP+	easy	4	✓	1931	Write a function to find minimum of three numbers. Your code must pass
135	Mbpps/232	MBPP+	easy	3	✓	1591	Write a function that takes in a list and an integer n and returns a l
136	Mbpps/233	MBPP+	easy	2	✓	1152	Write a function to find the lateral surface area of a cylinder. Your
137	Mbpps/234	MBPP+	easy	4	✓	1202	Write a function to find the volume of a cube given its side length. Y
138	Mbpps/235	MBPP+	easy	0			
139	Mbpps/237	MBPP+	easy	0			
140	Mbpps/238	MBPP+	easy	4	✓	426	Write a python function to count the number of non-empty substrings of
141	Mbpps/242	MBPP+	easy	3	✓	2163	Write a function to count the total number of characters in a string.
142	Mbpps/244	MBPP+	easy	0			
143	Mbpps/252	MBPP+	easy	3	✓	371	Write a python function to convert complex numbers to polar coordinate
144	Mbpps/253	MBPP+	easy	4	✓	259	Write a python function that returns the number of integer elements in
145	Mbpps/260	MBPP+	easy	0			

#	Problem	Pool	Level	Correct (of 4)	Kept	Kept thinking tokens	Question starts with
146	Mbpps/265	MBPP+	easy	0			
147	Mbpps/267	MBPP+	easy	0			
148	Mbpps/268	MBPP+	easy	0			
149	Mbpps/272	MBPP+	easy	2	✓	2353	Write a function that takes in a list of tuples and returns a list con
150	Mbpps/276	MBPP+	easy	2	✓	1217	Write a function that takes in the radius and height of a cylinder and
151	Mbpps/277	MBPP+	easy	4	✓	488	Write a function that takes in a dictionary and integer n and filters
152	Mbpps/280	MBPP+	easy	4	✓	302	Write a function that takes in an array and element and returns a tuple
153	Mbpps/282	MBPP+	easy	3	✓	1684	Write a function to subtract two lists element-wise. Your code must pa
154	Mbpps/283	MBPP+	easy	2	✓	667	Write a python function takes in an integer and check whether the freq
155	Mbpps/294	MBPP+	easy	0			
156	Mbpps/297	MBPP+	easy	2	✓	633	Write a function to flatten a given nested list structure. Your code m
157	Mbpps/305	MBPP+	easy	0			
158	Mbpps/309	MBPP+	easy	4			
159	Mbpps/311	MBPP+	easy	0			
160	Mbpps/312	MBPP+	easy	4	✓	267	Write a function to find the volume of a cone. Your code must pass thi
161	Mbpps/388	MBPP+	easy	1	✓	1411	Write a python function to find the highest power of 2 that is less th
162	Mbpps/389	MBPP+	easy	2	✓	714	Write a function to find the n'th lucas number. Your code must pass th
163	Mbpps/390	MBPP+	easy	0			
164	Mbpps/391	MBPP+	easy	0			
165	Mbpps/398	MBPP+	easy	0			
166	Mbpps/405	MBPP+	easy	2	✓	220	Write a function to check whether an element exists within a tuple. Yo
167	Mbpps/406	MBPP+	easy	3	✓	169	Write a python function to find whether the parity of a given number i
168	Mbpps/409	MBPP+	easy	1	✓	1185	Write a function to find the minimum product from the pairs of tuples
169	Mbpps/412	MBPP+	easy	4	✓	592	Write a python function to remove odd numbers from a given list. Your
170	Mbpps/413	MBPP+	easy	2	✓	750	Write a function to extract the nth element from a given list of tuple
171	Mbpps/414	MBPP+	easy	4	✓	399	Write a python function to check whether any value in a sequence exist
172	Mbpps/418	MBPP+	easy	4	✓	471	Write a python function to find the element of a list having maximum l
173	Mbpps/419	MBPP+	easy	2	✓	1203	Write a function to round every number of a given list of numbers and
174	Mbpps/420	MBPP+	easy	4	✓	723	Write a python function to find the cube sum of first n even natural n
175	Mbpps/421	MBPP+	easy	0			
176	Mbpps/422	MBPP+	easy	4	✓	340	Write a python function to find the average of cubes of first n natura
177	Mbpps/424	MBPP+	easy	3	✓	296	Write a function to extract only the rear index element of each string
178	Mbpps/425	MBPP+	easy	4	✓	447	Write a function to count the number of sublists containing a particul
179	Mbpps/426	MBPP+	easy	1	✓	1370	Write a function to filter odd numbers. Your code must pass this test:

#	Problem	Pool	Level	Correct (of 4)	Kept	Kept thinking tokens	Question starts with
180	Mbpps/428	MBPP+	easy	2	✓	583	Write a function to sort the given array by using shell sort. Your cod
181	Mbpps/430	MBPP+	easy	0			
182	Mbpps/435	MBPP+	easy	2	✓	473	Write a python function to find the last digit of a given number. Your
183	Mbpps/437	MBPP+	easy	2	✓	431	Write a function to remove odd characters in a string. Your code must
184	Mbpps/439	MBPP+	easy	2	✓	2569	Write a function to join a list of multiple integers into a single int
185	Mbpps/447	MBPP+	easy	4	✓	2063	Write a function to find cubes of individual elements in a list. Your
186	Mbpps/450	MBPP+	easy	4	✓	218	Write a function to extract specified size of strings from a given lis
187	Mbpps/454	MBPP+	easy	3	✓	2867	Write a function that matches a word containing 'z'. Your code must pa
188	Mbpps/455	MBPP+	easy	3	✓	494	Write a function to check whether the given month number contains 31 d
189	Mbpps/457	MBPP+	easy	3	✓	639	Write a python function to find the sublist having minimum length. You
190	Mbpps/459	MBPP+	easy	1	✓	930	Write a function to remove uppercase substrings from a given string. Y
191	Mbpps/460	MBPP+	easy	1	✓	627	Write a python function to get the first element of each sublist. Your
192	Mbpps/462	MBPP+	easy	0			
193	Mbpps/465	MBPP+	easy	2	✓	440	Write a function to drop empty items from a given dictionary. Your cod
194	Mbpps/470	MBPP+	easy	1	✓	515	Write a function to find the pairwise addition of the neighboring elem
195	Mbpps/471	MBPP+	easy	4	✓	442	Write a python function to find the product of the array multiplicatio
196	Mbpps/472	MBPP+	easy	4	✓	623	Write a python function to check whether the given list contains conse
197	Mbpps/475	MBPP+	easy	2	✓	389	Write a function to sort a dictionary by value. Your code must pass th
198	Mbpps/476	MBPP+	easy	4	✓	115	Write a python function to find the sum of the largest and smallest va
199	Mbpps/477	MBPP+	easy	2	✓	211	Write a python function to convert the given string to lower case. You
200	Mbpps/478	MBPP+	easy	3	✓	433	Write a function to remove lowercase substrings from a given string. Y
201	Mbpps/479	MBPP+	easy	2	✓	776	Write a python function to find the first digit of a given number. You
202	Mbpps/554	MBPP+	easy	2	✓	182	Write a python function which takes a list of integers and only return
203	Mbpps/556	MBPP+	easy	0			
204	Mbpps/558	MBPP+	easy	1	✓	3445	Write a python function to find the sum of the per-digit difference be
205	Mbpps/559	MBPP+	easy	0			
206	Mbpps/568	MBPP+	easy	2	✓	208	Write a function to create a list of N empty dictionaries. Your code m
207	Mbpps/569	MBPP+	easy	3	✓	535	Write a function to sort each sublist of strings in a given list of li
208	Mbpps/572	MBPP+	easy	0			
209	Mbpps/573	MBPP+	easy	4	✓	462	Write a python function to calculate the product of the unique numbers
210	Mbpps/577	MBPP+	easy	0			
211	Mbpps/580	MBPP+	easy	0			
212	Mbpps/581	MBPP+	easy	0			

#	Problem	Pool	Level	Correct (of 4)	Kept	Kept thinking tokens	Question starts with
213	Mbpps/585	MBPP+	easy	4	✓	197	Write a function to find the n most expensive items in a given dataset
214	Mbpps/587	MBPP+	easy	2	✓	2309	Write a function to convert a list to a tuple. Your code must pass thi
215	Mbpps/589	MBPP+	easy	0			
216	Mbpps/591	MBPP+	easy	0			
217	Mbpps/592	MBPP+	easy	0			
218	Mbpps/596	MBPP+	easy	2	✓	385	Write a function to find the size in bytes of the given tuple. Your co
219	Mbpps/599	MBPP+	easy	2	✓	416	Write a function to find sum and average of first n natural numbers. Y
220	Mbpps/602	MBPP+	easy	3	✓	395	Write a python function to find the first repeated character in a give
221	Mbpps/603	MBPP+	easy	0			
222	Mbpps/604	MBPP+	easy	2	✓	410	Write a function to reverse words seperated by spaces in a given strin
223	Mbpps/606	MBPP+	easy	4	✓	180	Write a function to convert degrees to radians. Your code must pass th
224	Mbpps/607	MBPP+	easy	0			
225	Mbpps/608	MBPP+	easy	1	✓	1865	Write a python function to find nth bell number. Your code must pass t
226	Mbpps/612	MBPP+	easy	1	✓	2173	Write a python function which takes a list of lists, where each sublis
227	Mbpps/615	MBPP+	easy	0			
228	Mbpps/616	MBPP+	easy	4	✓	303	Write a function which takes two tuples of the same length and perform
229	Mbpps/619	MBPP+	easy	0			
230	Mbpps/620	MBPP+	easy	0			
231	Mbpps/623	MBPP+	easy	1	✓	2906	Write a function to compute the n-th power of each number in a list. Y
232	Mbpps/629	MBPP+	easy	3	✓	369	Write a python function to find even numbers from a list of numbers. Y
233	Mbpps/632	MBPP+	easy	1	✓	437	Write a python function to move all zeroes to the end of the given lis
234	Mbpps/633	MBPP+	easy	0			
235	Mbpps/635	MBPP+	easy	3	✓	599	Write a function to sort the given list. Your code must pass this test
236	Mbpps/643	MBPP+	easy	0			
237	Mbpps/721	MBPP+	easy	0			
238	Mbpps/723	MBPP+	easy	2	✓	2626	The input is defined as two lists of the same length. Write a function
239	Mbpps/725	MBPP+	easy	1	✓	1253	Write a function to extract values between quotation marks " " of the
240	Mbpps/726	MBPP+	easy	3	✓	1698	Write a function that takes as input a tuple of numbers (t ₁ ,...,t _N +
241	Mbpps/728	MBPP+	easy	3	✓	304	Write a function takes as input two lists [a ₁ ,...,a _n], [b ₁ ,...,b _n]
242	Mbpps/732	MBPP+	easy	1	✓	640	Write a function to replace all occurrences of spaces, commas, or dots
243	Mbpps/733	MBPP+	easy	2	✓	656	Write a function to find the index of the first occurrence of a given
244	Mbpps/734	MBPP+	easy	0			
245	Mbpps/735	MBPP+	easy	0			
246	Mbpps/736	MBPP+	easy	3	✓	483	Write a function to locate the left insertion point for a specified va

#	Problem	Pool	Level	Correct (of 4)	Kept	Kept thinking tokens	Question starts with
247	Mbpps/737	MBPP+	easy	4	✓	424	Write a function to check whether the given string is starting with a
248	Mbpps/739	MBPP+	easy	0			
249	Mbpps/741	MBPP+	easy	3	✓	569	Write a python function to check whether all the characters are same o
250	Mbpps/742	MBPP+	easy	0			
251	Mbpps/744	MBPP+	easy	4	✓	251	Write a function to check if the given tuple has any none value or not
252	Mbpps/745	MBPP+	easy	0			
253	Mbpps/749	MBPP+	easy	0			
254	Mbpps/753	MBPP+	easy	3	✓	420	Write a function to find minimum k records from tuple list. https://www
255	Mbpps/754	MBPP+	easy	2	✓	647	We say that an element is common for lists l1, l2, l3 if it appears in
256	Mbpps/757	MBPP+	easy	0			
257	Mbpps/758	MBPP+	easy	0			
258	Mbpps/760	MBPP+	easy	3	✓	273	Write a python function to check whether a list of numbers contains on
259	Mbpps/762	MBPP+	easy	4	✓	3716	Write a function to check whether the given month number contains 30 d
260	Mbpps/763	MBPP+	easy	0			
261	Mbpps/766	MBPP+	easy	2	✓	2491	Write a function to return a list of all pairs of consecutive items in
262	Mbpps/767	MBPP+	easy	4	✓	456	Write a python function to count the number of pairs whose sum is equa
263	Mbpps/770	MBPP+	easy	4	✓	456	Write a python function to find the sum of fourth power of first n odd
264	Mbpps/771	MBPP+	easy	0			
265	Mbpps/772	MBPP+	easy	4	✓	436	Write a function to remove all the words with k length in the given st
266	Mbpps/773	MBPP+	easy	0			
267	Mbpps/775	MBPP+	easy	2	✓	627	Write a python function to check whether every odd index contains odd
268	Mbpps/778	MBPP+	easy	0			
269	Mbpps/780	MBPP+	easy	0			
270	Mbpps/784	MBPP+	easy	0			
271	Mbpps/785	MBPP+	easy	0			
272	Mbpps/786	MBPP+	easy	0			
273	Mbpps/787	MBPP+	easy	2	✓	541	Write a function that matches a string that has an a followed by three
274	Mbpps/790	MBPP+	easy	0			
275	Mbpps/793	MBPP+	easy	3	✓	882	Write a python function to find the last position of an element in a s
276	Mbpps/794	MBPP+	easy	0			
277	Mbpps/796	MBPP+	easy	3	✓	161	Write function to find the sum of all items in the given dictionary. Y
278	Mbpps/798	MBPP+	easy	4	✓	91	Write a python function to find the sum of an array. Your code must pa
279	Mbpps/807	MBPP+	easy	2	✓	294	Write a python function to find the first odd number in a given list o
280	Mbpps/808	MBPP+	easy	4	✓	2887	Write a function to check if the given tuples contain the k or not. Yo